第 8 章 深度学习

章节导航

- 了解深度学习的发展历程、常见任务与编程框架。
- 熟悉卷积神经网络和循环神经网络的经典算法的原理，掌握其在图像识别和自然语言处理方面的应用。
- 以食物图像鉴别为例开展项目实践，掌握数据准备、网络搭建、模型训练、测试和优化改进的全流程操作。

本章知识导图

8.1 深度学习概述

8.1.1 人工智能与神经网络

当提及人工智能时，一般公众往往首先联想到影视作品中的形象。例如，2001 年在电影《人工智能》中，出现了与人类外貌和智力相媲美的机器人；又如，在《终结者》系列电影中出现了具备机械骨骼和人类组织的机器人，以及超越普通人类肉体和智力的具有自我修复功能的液态金属智能体。这些对人工智能的大胆想象给人们留下了深刻的印象。

人工智能算法历史

近年来，人工智能在科技领域取得了显著成效，这一点是不言而喻的。从 AlphaGo 战胜围棋世界冠军李世石到无人驾驶技术的成功实践，这些引人注目的成就吸引了大众广泛的关注。作为人工智能的一个分支，机器学习受到了人们广泛的关注，其中深度学习成为近年的研究热点之一。

1. 人工智能发展史

图 8-1 展示了人工智能的发展历程及其主要特点，下面对其进行简要介绍。

人工智能发展历史

1956 年，在达特茅斯会议上首次提出了人工智能这一概念 9，参加这次会议的专家学者众多，包括人工智能概念的提出者约翰·麦卡锡(John Mac Carthy)，信息论的创始人克劳德·香农(Claude Shannon)，第十届诺贝尔经济学奖得主赫伯特·西蒙(Herbert Simon)，图灵奖(1969 年)获得者马文·明斯基(Marvin Minsky)，机器学习之父阿瑟·萨缪尔(Arthur Samuel)等。1956 年被称为人工智能元年。人工智能的发展也由此拉开序幕。

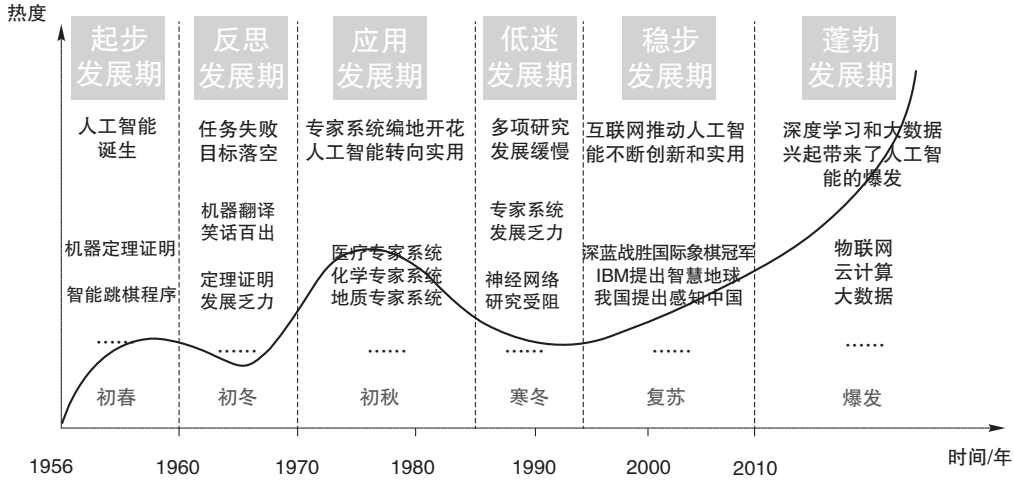


图 8-1　人工智能的发展历程及其主要特点

1957 年，弗兰克·罗森布拉特(Frank Rosenblatt)提出感知机模型，这是人工神经网络(artificial neural network，ANN)理论中关于神经元的最早模型。即使在深度学习火热发展的今天，为了理解神经元的本质，很多学者会将深度学习模型与基于感知机的模型进行综合比较与分析。正因为感知机模型被提出并受到热捧，各类人工神经网络理论在各个领域也实现了重大突破，人工智能受到了很多学者广泛的关注，在此期间很多国家投入大笔资金开展了许多相关项目的研究。

1974—1980 年，感知机模型无法解决现实中复杂多变的问题，人们过度高估了科学技术的发展速度，导致很多学者开始怀疑人工智能，其遇到了第一次寒冬。著名数学家莱特希尔(Lighthill)向英国政府提交了一份关于人工智能的研究报告，其中对当时的机器人技术、语言处理技术和图像识别技术进行了严厉的批评，尖锐地指出人工智能中那些看上去宏伟的目标根本无法实现。此后，科学界对人工智能进行了一轮深入的拷问，使人工智能的发展陷入困境，人们对其实际价值产生了诸多质疑。计算能力有限、缺乏可用数据成为当时阻碍人工智能发展的两大瓶颈。

1980—1987 年，随着专家系统的问世和误差反向传播(back propagation，BP)算法的提出，人工智能又开启了新的发展篇章。专家系统是一种程序，依据一组从专门知识中推演出的逻辑规则，能够在某一特定领域回答问题或解决问题。1978 年，卡耐基梅隆大学开始开发一款能够帮助顾客自动选配计算机配件的程序软件 XCON，该软件在 1980 年投入使用，其中包含设定好的 2 500 多条规则，在使用期间处理了超过 80 000 条订单，准确度超过 95%，每年节省资金超过 2 500 万美元。在这之后，各个国家纷纷拨款投资这种类型的项目，希望制造出能够与人对话、翻译语言、解释图像并且像人一样具备推理能力的机器。人工智能又迎来了大发展。1982 年，霍普菲尔德(Hopfield)提出一种新型的神经网络，后来被称为 Hopfield 网络，它使用一种全新的方式学习和处理信息。BP 算法在此期间被提出，这种算法也是深度学习理论中用于神经网络训练的重要算法之一，沉寂多年的人工神经网络方向的研究重新获得发展。

但不幸的是，人工智能又在 1987—1993 年迎来第二次寒冬。专家系统最初取得的成功

是有限的，它无法自我学习并更新知识库和算法，维护所需的费用与时间越来越多，以至于很多企业后来都放弃了陈旧的专家系统。理论研究的迷茫加上各国政府和机构纷纷停止向人工智能研究领域投入资金，导致人工智能又一次跌入低谷。

1993 年，人们开始重新思考人工智能，认为使用人类设定的规则进行编程不符合人工智能的本质，他们认为这种方法本质上是错误的。大象不懂象棋游戏的规则，但大象可以从现实中学会识别环境并做出判断，人工智能技术也应该拥有身体感知能力，从基础感知到大脑认知的从下而上的过程才能实现真正的智能。这种观点可能在当时的科技水平下显得有些超前，但实实在在地推动了后续神经网络技术的壮大和发展。

随着人工智能技术的发展，尤其是神经网络技术的逐步发展，以及人们对人工智能的认知趋于客观、理性，人工智能技术开始进入平稳发展时期。1997 年 5 月 11 日，IBM 的计算机系统“深蓝”战胜了国际象棋世界冠军加里·卡斯帕罗夫(Garry Kasparov)，又一次在公众领域引发了现象级的人工智能话题讨论。这是人工智能发展的一个重要里程碑。

2006 年，被誉为“深度学习之父”的辛顿(Hinton)在神经网络的深度学习领域取得突破，其在反向传播算法、深度信念网络、卷积神经网络等领域做出了重要贡献，人类又一次看到机器赶超人类的希望，这是标志性的技术进步。

2011 年，IBM 开发的人工智能程序沃森(Watson)参加了一档智力问答节目并战胜了两位人类冠军。沃森存储了 2 亿页数据，能够将与问题相关的关键词从看似相关的答案中抽取出来。这一人工智能程序已被 IBM 广泛应用于医疗诊断领域。

2016—2017 年，AlphaGo 战胜围棋世界冠军李世石。AlphaGo 是由谷歌 DeepMind 开发的人工智能围棋程序，具有自我学习能力。它能够搜集大量围棋对弈数据和棋谱，学习并模仿人类下棋。

2017 年，深度学习大热。AlphaGo Zero(第四代 AlphaGo)在无任何数据输入的情况下，开始自学围棋，3 天后便以 100∶0 横扫了第二个版本“旧狗”，学习 40 天后，又战胜了在人类高手看来不可企及的第三个版本“大师”。

至今，人工智能的发展仍在继续。在人工智能的发展过程中，机器学习及神经网络两大概念的地位突出。当前已进入大模型的迅猛发展期，大数据、大算力和大算法完美结合，大幅提升了大模型的预训练和生成能力及多模态、多场景应用能力。例如，ChatGPT 的巨大成功，就是在微软 Azure 强大的算力及维基等海量数据的支持下，在 Transformer 架构基础上，坚持 GPT 模型及人类反馈的强化学习(reinforcement learning from human feedback，RLHF)进行精调的策略下取得的。同时，我国科技龙头企业密集发布自研大模型，例如，百度发布大模型文心一言，阿里发布首个超大规模语言模型通义千问，腾讯混元 AI 大模型团队推出了万亿级别中文 NLP 预训练模型 HunYuan-NLP-1T。华为发布的盘古大模型是业界首个千亿级生成和理解中文 NLP 大模型。

2. 人工神经网络

顾名思义，机器学习的目的是使计算机像人类一样学习和行动，通过观察现实世界的信息并进行演绎，基于一定时间的知识积累以自主方式提高计算机的学习能力。

实现机器学习可以分成两步：训练(归纳，induction)和预测(演绎，deduction)。

(1)归纳。归纳是指从具体的事实归纳总结出一般性规律。机器学习中的训练是给定一

定数量的样本(已知模型输入 X 和模型输出 Y)中，学习输出 Y 与输入 X 的关系(可以想象成某种映射)。

(2)演绎。演绎是指从一般性规律推导出具体案例的结果，是实际运用模型的过程。机器学习中的预测是基于训练得到的 Y 与 X 之间的关系，计算新样本对应的输出 Y。如果通过模型计算的输出和真实场景的输出一致，则说明模型是有效的。

通过机器学习，可以大大减少人工成本。不难预见，机器学习将是推动第四次工业革命的主力军。机器学习的实现主要通过人工神经网络完成，人工神经网络是一种模仿生物神经网络的结构和功能的数学模型或计算模型。神经网络由大量的人工神经元联结进行计算。大多数情况下，人工神经网络能在外界信息的基础上改变内部结构，是一种自适应系统。神经网络重要的用途之一是实现分类任务，例如，运用神经网络，可以实现垃圾邮件识别、图片识别等功能。

8.1.2　神经网络的深度

既然神经网络的神经元(节点)越多，模型的表达能力越强，模型的预测效果越好，那么为什么深度学习被广泛接受，而不是宽度学习呢？本节将介绍深度神经网络的发展历程，然后解释选择深度神经网络的原因，并探究神经网络是否越深越好。

1. 深度神经网络的发展历程

神经网络模型最早起源于认知心理学、理论和计算神经科学领域。1943 年，心理学家沃伦·麦卡洛克(Warren McCulloch)与数理逻辑学家沃尔特·皮茨(Walter Pitts)在合作的论文 *A logical calculus of the ideas immanent in nervous activity* 中提出了神经网络的概念与人工神经元的数学模型，即 MCP 神经元数学模型，为神经网络的研究拉开了序幕。

第一代神经网络能够对如三角形、四边形等简单形状进行分类。这也使得人们逐渐认识到计算机或许可以借此实现类似于人类的学习、记忆、识别能力。然而，第一代神经网络并不完善，当时的感知机特征提取层甚至可能还需要手动调节参数才能有效工作，这违背了其智能的要求，因此神经网络的发展停滞了相当长的一段时间。

1985 年，被誉为“神经网络之父”的辛顿提出了一种新的思想——即使用多个隐藏层来取代原先感知机中的单个特征层，并利用 BP 算法计算网络参数。这一想法使得神经网络的发展经历了一次复兴。1989 年，杨立昆(Yann LeCun)等利用深度神经网络来识别手写体邮政编码字符，进一步运用卷积神经网络(convolutional neural network，CNN)成功实现了银行支票上手写体字符的识别，其准确率达到了商用级别。尽管卷积神经网络取得了显著的成果，但在小型数据集的训练上仍需要耗费长达 3 天的时间。随后，出现了结构相对简单且训练速度更快的传统方法，如支持向量机、随机森林，使得神经网络的研究再次进入寒冬。

2011 年，得益于辛顿和本吉奥(Bengio)等的研究，深度神经网络在图像和语音识别领域取得突破性成果，受到人们的重视，再次掀起了深度神经网络的研究热潮。之后亚历克斯·克里切夫斯基(Alex Krizhevsky)等提出将线性整流(rectified linear unit，ReLU)函数作为卷积神经网络的激活函数，使得神经网络在图像识别任务中取得巨大优势，开始了神经网络快速发展的新历程。

2. 选择深度神经网络的原因

首先，需要了解深度神经网络与以前神经网络的区别。简而言之，在深度神经网络中，网络的层数较多，使用了多个隐藏层。2012 年，深度学习和卷积神经网络开始受到人们的广泛关注，当时 ImageNet 竞赛的冠军方案只使用了 8 层神经网络。如今，层数达到数百层甚至数千层的深度网络越来越常见，这得益于计算机算力的提升。可以得出结论：模型的复杂度越高，其学习能力越强。那么如何增加模型的复杂度？对于神经网络模型而言，有两种方法：一是增加模型的深度，二是增加模型的宽度。当模型变宽时，只是增加了计算单元的数量，增加了函数的个数。然而，当模型变深时，不仅增加了神经元的数量，还增加了嵌入的层级。因此，应选择深度神经网络而不是宽度神经网络。

基于以上观点，既然深度神经网络可以增强模型的学习能力，那么网络深度越深，其效果是否越好？答案是否定的，这是因为在机器学习过程中存在一个重要问题——过拟合。通过机器学习，希望获得事物的一般规律，但有时过强的学习能力会导致模型学习到训练数据本身的一些特征和特性，并错误地将其当作一般规律使用，从而产生不可预测的错误。幸运的是，目前已经有许多方法可以缓解过拟合问题，例如使用计算能力强大的设备进行大规模数据训练。

8.1.3 深度学习的常见任务

深度神经网络的两大任务分别是自然语言处理及计算机视觉。

自然语言处理的发展主要分为 4 个阶段。第一个阶段是 1956 年之前的萌芽期。1936 年，图灵首次提出“图灵机”的概念，将人们使用纸和笔进行数学运算的过程进行抽象，由一个虚拟的机器来替人类进行数学运算，正是这一理论基础促使了 1946 年电子计算机的诞生。同时，电子计算机的诞生又为机器翻译与自然语言处理提供了硬件基础。1957—1970 年，自然语言处理进入快速发展期。在这个时期，自然语言处理融入人工智能领域的研究，并基于研究分为两大阵营：符号派与随机派。1967 年，美国心理学家奈瑟尔(Neisser)提出认知心理学的概念，直接将自然语言处理与人类认知联系起来。但是，1971—1993 年，自然语言处理的研究进入“冰河时期”。随着研究的深入，人们发现基于自然语言处理的应用不能在短时间内得到解决，新的问题却不断涌现。在这个时期，虽然还有一些发达国家的研究人员仍对此进行不懈研究，却收效甚微。1994 年之后，对自然语言处理的研究重新进入复苏融合期，计算机的处理速度及存储容量大幅提升，将自然语言处理推上了更高的平台，使得基于语音和语言处理的商品化开发成为可能。

深度神经网络中的另一大任务——计算机视觉，包含图像分类、目标检测、语义分割、实例分割、目标追踪等任务。在图像分类中，人脸表情识别是目前热门的研究方向之一，人脸表情识别的主要目标是根据给定的面部图像识别人类的情绪状态(如愤怒、蔑视、厌恶、恐惧、快乐等)。在目标检测任务中，涌现出诸多改进的检测算法并应用在现实场景中。在分割任务中，图像识别应用在医学图像场景中，并成为计算机辅助诊断的得力助手，曾格(Zeng)等利用深度信念网络(deep belief network)对胶体金免疫层析试纸条图像中的质控线和测试线进行分割来判断疾病的阴阳性，并进一步进行定量分析。

在深度学习的研究中，除了上述的计算机视觉和自然语言处理两大任务，深度学习还在推荐系统、智能控制、自动驾驶等任务处理中展现出其强大的威力，在交通物流、工业质检、金融行业、能源电力、智慧农业等领域已实现产业化应用。

8.1.4　深度学习的利器

本节将介绍深度学习编程实现中常用的框架，包括 TensorFlow、PyTorch、Caffe、PaddlePaddle 等。同时，简要介绍深度学习常用的 Python 库，如 NumPy、Pandas、scikit-learn 等。此外，还会介绍一些常见的深度学习开源代码网站。

使用 PyTorch 搭建神经网络

1. 常见的框架

1)TensorFlow

TensorFlow 是一个基于数据流编程(dataflow programming)的符号数学系统，被广泛应用于各类机器学习算法的编程实现，其前身是谷歌的神经网络算法库 DistBelief。安装 TensorFlow，可以在计算机终端直接使用模块管理工具 pip/pip3 或 conda。下面是使用两种工具的安装命令。

```
pip install tensorflow                    # 选择一种安装方式
conda install -c conda-forge tensorflow   # 选择另一种安装方式
```

2)PyTorch

PyTorch 是一个开源的 Python 语言机器学习库，由 Facebook(2021 年更名为 Meta)人工智能研究院于 2017 年 1 月推出。PyTorch 提供两个高级功能：一是使用图形处理单元(graphics processing unit，GPU)加速张量计算；二是提供自动求导功能。PyTorch 入门简单，其设计追求最少的封装，对初学者友好，也是深度学习的一个利器。

PyTorch 的安装十分简单，根据 PyTorch 官网的提示，选择合适的系统及安装方式即可。这里以 Anaconda 为例，介绍如何安装 PyTorch 并配置开发环境。

```
# CUDA 12.1
pip3 install torch==2.2.2 torchvision==0.17.2 torchaudio==2.2.2--index-url https://
download.pytorch.org/whl/cu121

# CUDA 11.8
pip3 install torch==2.2.2 torchvision==0.17.2 torchaudio==2.2.2--index-url https://
download. pytorch.org/whl/cu118

# CPU 版本
pip3 install torch==2.2.2 torchvision==0.17.2 torchaudio==2.2.2 --index-url https://
download.pytorch.org/whl/cpu
```

安装完成后，可以导入 Torch 并查看其版本号。

```
import torch
torch.__version__
```

新建的环境是没有安装 ipykernel 的，无法注册到 Jupyter Notebook 中，所以先要准备 PyTorch 的开发环境。

```
# 安装 ipykernel
conda install ipykernel
# 写入环境
python -m ipykernel install --name pytorch --display-name "Pytorch for Deeplearning"
```

下一步要创建 Jupyter Notebook 配置文件。

```
# 切换回基础环境
activate base
# 创建 jupyter notebook 配置文件
jupyter notebook --generate-config
# 显示创建 jupyter_notebook_config.py 的具体位置
```

打开创建的配置文件，将下面的代码写入文件末尾。

```
c.ServerApp.notebook_dir=''    # 默认目录位置
c.ServerApp.iopub_data_rate_limit=100000000    # 这个改大一些否则有可能报错
```

至此，PyTorch 的开发环境配置完成，打开 Jupyter Notebook，在 New 按钮的下拉菜单中选择创建文件命令时，选择 Pytorch for Deeplearning 环境，就可以创建 PyTorch 的相关开发环境了。

3）Caffe

Caffe(convolutional architecture for fast feature embedding)是一个兼具表达性、速度和思维模块化的深度学习框架，由伯克利人工智能研究小组和伯克利视觉与学习中心开发。虽然其内核是用 C＋＋语言编写的，但 Caffe 有 Python 和 MATLAB 的相关接口。Caffe 支持多种类型的深度学习架构、面向图像分类和图像分割，还支持 CNN、R-CNN、长短期记忆网络(long short-term memory network，LSTM)和全连接神经网络设计。Caffe 完全开源，同时提供了一个用于训练、测试的完整工具包，可以帮助使用者快速上手。

4）PaddlePaddle

PaddlePaddle(飞桨)以百度多年的深度学习技术研究和业务应用为基础，集深度学习核心训练和推理框架、基础模型库、端到端开发套件、丰富的工具组件于一体，是我国首个自主研发、功能完备、开源开放的产业级深度学习平台。

2. 常见的 Python 库

1）Pandas

Pandas 库是一个用于数据处理和分析的强大 Python 库。它提供了两种主要的数据结构：Series 和 DataFrame。其中，Series 是一个一维标记数组，类似于带有标签索引的 NumPy 数组，可以包含任何数据类型，它类似于 Python 语言中的字典或列表。DataFrame 是一个二维数据结构，类似于电子表格或 SQL 表格。DataFrame 由行和列组成，并且可以

包含不同类型的数据。DataFrame 可以看作 Series 的容器。

Pandas 库提供了丰富的数据操作和转换功能，如数据筛选、切片、聚合、合并、拆分等。它能够快速地读取、写入各种数据源，如 CSV 文件、Excel 文件、数据库文件等。Pandas 库广泛用于数据清洗、数据预处理和数据分析任务。

2）NumPy

NumPy 库是 Python 语言中科学计算的核心库。它提供了高性能的多维数组对象（称为 ndarray），以及用于在这些数组上执行操作的函数。NumPy 库的核心功能如下。

（1）多维数组。NumPy 库的 ndarray 是一个快速且灵活的数组对象，能够容纳同类型的数据。它使得数组操作更加高效和方便。

（2）广播（broadcasting）。NumPy 库允许不同形状的数组在一些条件下进行数学运算，而无须进行显式循环，这称为广播。

（3）数学函数。NumPy 库提供了许多数学函数，如 sin( )、cos( )、exp( )等，以及线性代数、随机数生成等功能。

NumPy 库广泛应用于数据科学、机器学习和科学计算领域，它的数组操作速度快且支持广播功能，使得它在大规模数据计算上非常高效。

3）scikit-learn

scikit-learn（简称 sklearn）库是 Python 语言中非常受欢迎的机器学习库之一，它提供了丰富的机器学习算法和工具，用于处理分类、回归、聚类、降维、模型选择和预处理等任务。scikit-learn 库的特点如下。

（1）一致的 API。sklearn 提供了简单一致的 API 接口，方便用户使用不同的机器学习算法，并进行模型的训练和评估。

（2）丰富的算法。sklearn 包含大量的机器学习算法，如 SVM、随机森林、k 近邻、逻辑回归等。

（3）数据预处理。sklearn 提供了许多数据预处理方法，如特征缩放、数据标准化、特征选择等，用于优化模型性能。

（4）评估工具。sklearn 包含各种评估指标和交叉验证技术，用于评估模型的性能和泛化能力。

sklearn 广泛应用于各种机器学习项目，从小规模研究项目到大规模生产环境的应用都能得到该库的支持。

3. 常见的深度学习开源代码网站

Papers with Code 创立于 2018 年 7 月，旨在帮助广大机器学习科研人员和业界人士跟踪最新发布的论文及源代码，使他们快速了解最前沿的技术进展。该平台涉及各类机器学习任务和细分方向。截至 2024 年，Papers with Code 已经累积了超过 120 000 篇论文、超过 4 000 项任务和超过 10 500 个排行榜，其已成长为非常常用的机器学习资源网站之一（https://paperswithcode.com/）。

使用 PyTorch 搭建神经网络

机器之心是我国领先的前沿科技媒体和产业服务平台，关注人工智能、机器人和神经认知科学，坚持为从业者提供高质量内容和多项产业服务。机器之心的 SOTA 网站可以视作国内版的 Papers with Code，用户可以根据自己的需要寻找机器学习对应领域和任务下的 SOTA 论文，

平台会提供论文、模型、数据集和 benchmark 的相关信息(https://sota.jiqizhixin.com/)。

Model Zoo 为深度学习研究人员提供了一个平台，可以轻松地为各种深度学习框架和用途找到经过预训练的模型。Model Zoo 会定期更新 GitHub 站点，并为用户提供筛选功能，以查找用户所需的模型(https://www.modelzoo.co/)。

8.2 卷积神经网络

8.2.1 基础知识

本节首先说明什么是卷积神经网络及怎样进行卷积操作，接着介绍单通道卷积、多通道卷积等概念，然后介绍池化(pooling)、填充(padding)、激活函数等基础知识。

卷积神经网络是一种特殊结构的神经网络，在与数字图像相关的任务中表现优异，其核心是可以有效提取图像特征的卷积操作。卷积操作的过程如图 8-2 所示。

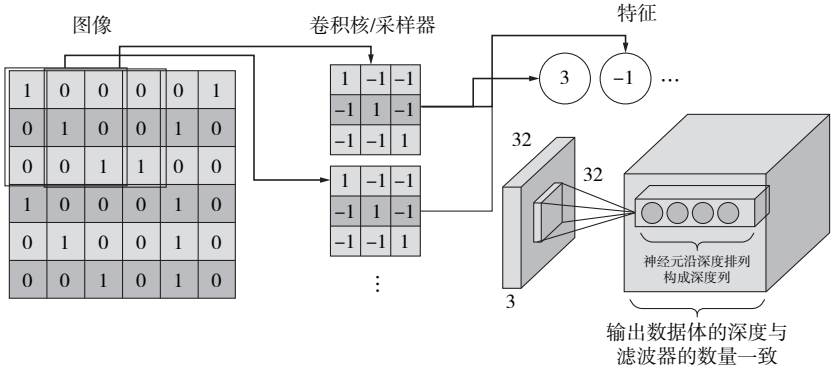


图 8-2　卷积操作的过程

1. 单通道卷积

在深度学习中，卷积操作是逐元素先乘法后加法的运算。对于具有一个通道的图像，卷积示意图如图 8-3 所示。这里的卷积核是一个 3×3 矩阵，即[[0，1，2]，[2，2，0]，[0，1，2]]，卷积核在输入端滑动。在每个位置，它都在进行元素乘法和加法。每个滑动位置最终得到的结果为逐元素相乘后的和，输出是 3×3 的特征矩阵。

卷积神经网络可视化

图 8-3　单通道卷积示意图

2. 多通道卷积

在许多现实应用中，需要处理的是具有多个通道的彩色图像。典型的例子是 RGB 图像，每个通道分别表达了原始图像在红、绿、蓝 3 个颜色分量上的值。

图 8-4 给出了多通道卷积的运算过程。以彩色图像为例，包含的 3 个通道分别表示 RGB 三原色的像素值。输入层是一个 5×5×3 的矩阵，卷积核通道数为 3，每个通道的卷积核是一个 3×3 矩阵。卷积过程如下：首先每个通道的像素值与其对应的卷积核通道数值进行卷积，每个通道获得一个卷积结果，再将 3 个对应位置的卷积结果进行累加求和，最后得到卷积结果。由此可以发现：多通道卷积最终得到的结果是各个通道上的信息综合。

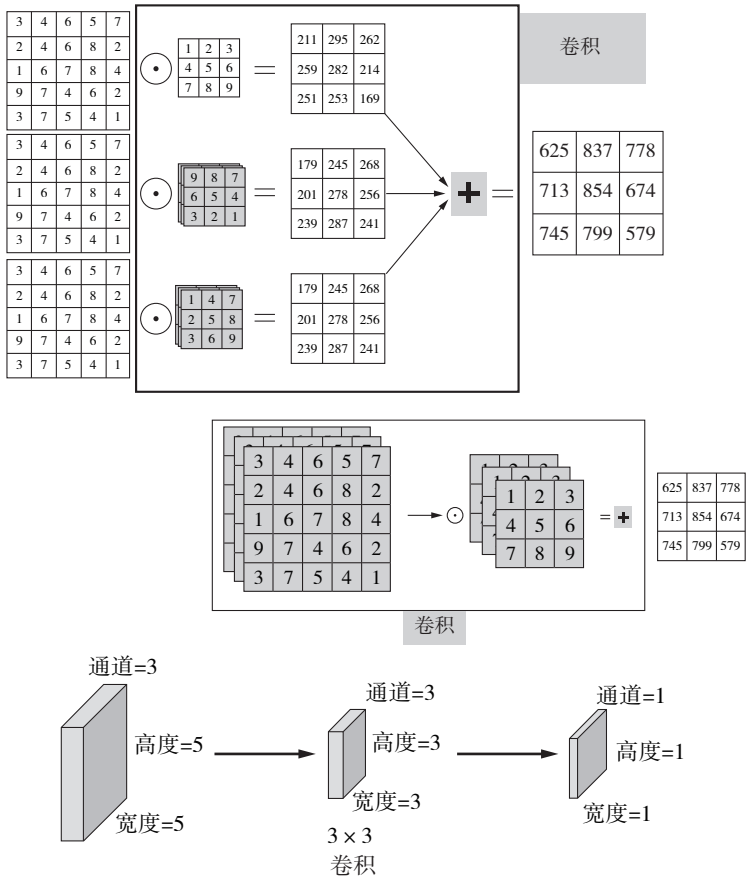


图 8-4　多通道卷积的运算过程

3. 多通道输出

在多通道卷积中，因为只有一个卷积核，所以卷积结果只有一个通道，从而只得到一种特征。若要卷积后的特征图中包含更多特征，就需要更多的特征通道，此时，只需增加卷积核的数量再进行通道拼接操作，输出的通道数即为卷积核的数量，如图 8-5 所示。

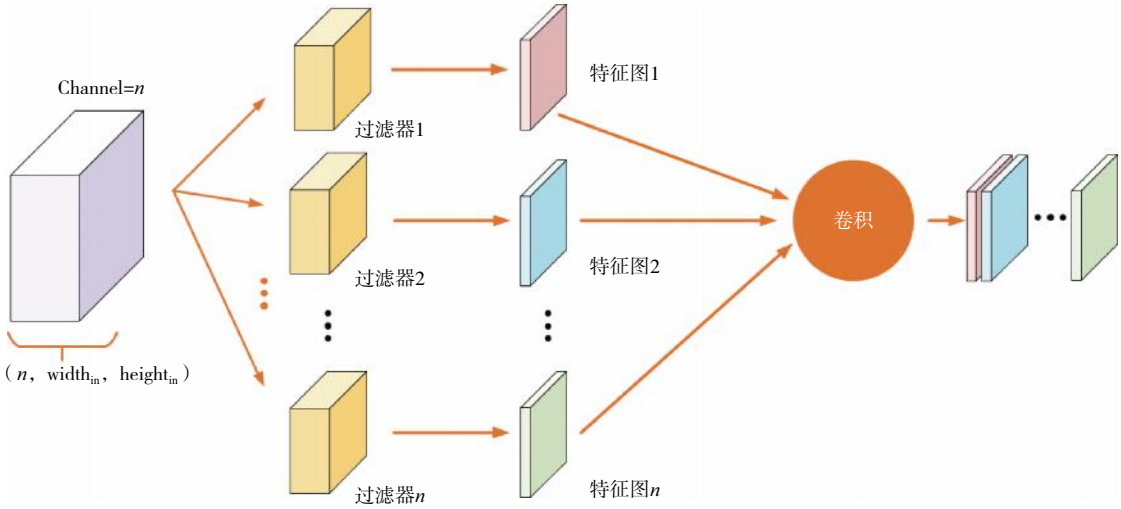


图 8-5　多通道输出示意图

4. 批次

当数据集规模较小时，可以使用全部数据更新网络参数。然而，当数据集规模变大时，若一次性使用所有训练样本来更新参数，则需要很大的内存来一次性载入所有数据，而且需要更多的计算资源来处理数据。在大数据时代，现有硬件条件难以满足这一要求。因此，每次可以随机选取一部分数据来更新参数，直至所有数据都参与训练，便完成了一轮(epoch)训练，通常训练一个网络需要上百轮甚至上千轮训练。上述过程中，每次随机选取的数据就称为一个批次(batch)，选取的数据量称为批量大小(batch size)。在实际训练模型的过程中，通常根据数据的维度和显存及内存的大小，选择一个适中的批量大小，在合理范围内增大它可以有效提高内存利用率，能够有效地加快算法的处理速度，但要注意的是，如果盲目增大它可能会使内存容量溢出。

5. 池化

池化层(pooling layer)通常设置在连续的卷积层中间，用于压缩数据与参数的量，减少过拟合，即池化层在保留数据主要特征的同时可以减少参数(降维)和计算量，提高模型的泛化能力，并且池化还能够刻意保持某种不变形(如旋转、平移、伸缩等)。例如，当输入为图像时，池化层最主要的作用就是压缩图像。池化方式主要包括平均池化和最大池化。

平均池化(mean pooling)按照固定大小的窗口进行分割，窗口的值取窗口内所有像素的平均值。平均池化能很好地保留整体数据的特征，且有利于突出背景信息。

最大池化(max pooling)通过取窗口中像素的最大值实现，其能够有效地保留纹理特征。一般情况下，大多数卷积网络使用最大池化。

图 8-6 是按 2×2 的网格进行平均池化和最大池化的结果。

6. 填充

填充操作是卷积神经网络中的一项基本操作。在卷积过程中，当使用一个 3×3 的卷积核对一个 6×6 的图像进行卷积时，特征图的输出尺寸将缩小为 4×4，即图像经过卷积操作后尺寸从 6×6 缩小到 4×4。在多次卷积操作后，图像的尺寸可能会进一步缩小至 1×1。

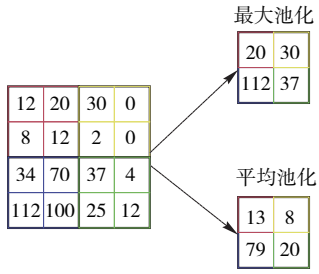


图 8-6　平均池化和最大池化的结果

从上述对卷积操作的理论分析可以得知，在图像边缘的像素点上，每个像素点只被一个输出所利用。这意味着在处理图像时，会丢失许多边缘位置的信息。为了解决上述问题，可以在卷积操作之前进行填充操作。上述例子中，可以在 6×6 的图像边缘添加额外的像素，使图像的尺寸变为 8×8，然后再进行卷积操作。通过填充操作，输出的特征图仍然保持 6×6 的尺寸，从而保持了原始输入图像的大小，并能够充分利用特征图的边缘信息。图 8-7 为填充操作示意图。

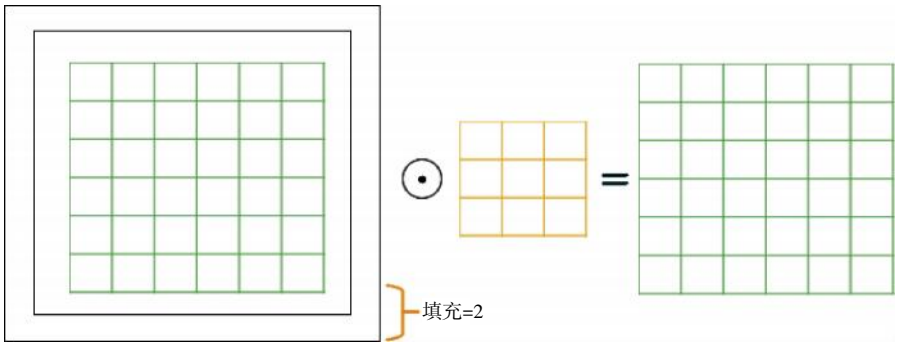


图 8-7　填充操作示意图

7. 激活函数

在实际应用中数据的分布大多是非线性的，然而一般神经网络的计算却是线性的。引入非线性激活函数可以较好地解决这一矛盾，这也是神经网络中强化网络学习能力的一种高效途径。下面列举了常用的激活函数的表达式。

Sigmoid 函数：

$$f(x)=\frac{1}{1+\mathrm{e}^{-x}}$$

双曲正切函数：

$$f(x)=\frac{\mathrm{e}^{x}-\mathrm{e}^{-x}}{\mathrm{e}^{x}+\mathrm{e}^{-x}}$$

Softmax 函数：

$$f(x)=\frac{\mathrm{e}^{x_i}}{\sum_i \mathrm{e}^{x_i}}$$

ReLU 函数：

$$f(x)=\max(0,\ x)$$

8.2.2 经典的卷积神经网络

卷积神经网络发展至今，已经出现了各式各样的网络结构，特别是在图像识别、图像检测和图像分割方面，不断出现新颖的卷积神经网络模型。下面详细介绍 5 种具有代表性的卷积神经网络。

1. LeNet5

20 世纪 90 年代，杨立昆等提出了用于手写数字和机器打印字符图片识别的神经网络，并将其命名为 LeNet5。作为卷积神经网络的开创性工作，该模型提出了三大思想，分别是局部感知、下采样、权值共享。

LeNet5 共有 7 层(不包含输入层)，每层都包含可训练参数，其网络模型架构如图 8-8 所示，输入为一张手写字体的图像，经过卷积层→池化层→卷积层→池化层，后接两个全连接层，最后的输出层是输入图像属于每个类别的概率。

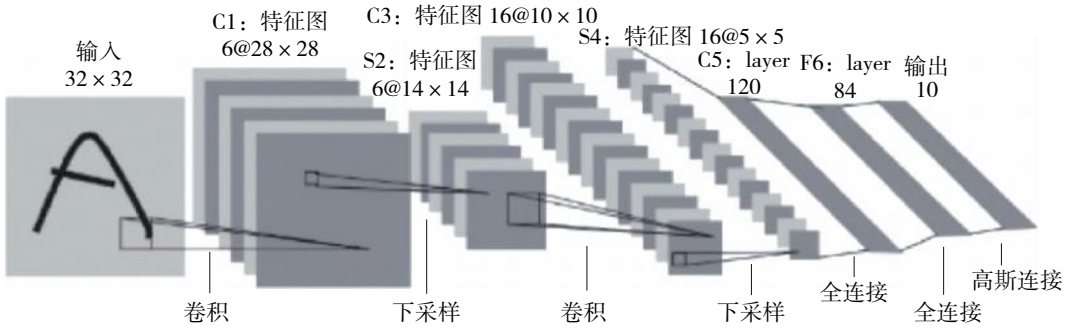


图 8-8　LeNet5 网络模型架构

下面详细介绍 LeNet5 网络模型架构。

输入层：LeNet5 网络的输入为 32×32 大小的图像，比后文用到的 MNIST 数据集的图像稍微大一些。

卷积层(C1)：使用 6 个大小为 5×5 的卷积核对输入图像进行卷积操作，卷积后得到的特征图的尺寸为 28×28，因此输出 6 个大小为 28×28 的特征图。

池化层(S2)：这里采用最大池化操作，池化窗口大小为 2×2。经过池化层后输出 6 个 14×14 的特征图，作为下一层的输入。

卷积层(C3)：使用 16 个大小为 5×5 的卷积核对输入的特征图进行卷积操作。值得注意的是，C3 层输出的特征图是由 S2 层中特征图加权组合得到的。输出为 16 个 10×10 的特征图。

池化层(S4)：同样采用最大池化操作，池化窗口大小为 2×2。最后输出 16 个 5×5 的特征图，神经元个数为 400(16×5×5)。

卷积层(C5)：该层是一个全连接的卷积层，使用 120 个 5×5 的卷积核对 S4 层的 16 个 5×5 的特征图进行卷积操作，得到 120 个的 1×1 特征图。

全连接层(F6)：该层有 84 个神经元，与 C5 层的 120 个特征图全连接。每个神经元接收来自 C5 层 120 个特征图的输入。

输出层：LeNet-5 的最后一层是一个具有 10 个神经元的输出层。该层采用 softmax 激活函数，输出一个概率分布结果，用于分类。

2. AlexNet

受计算机性能的影响，虽然 LeNet5 网格模型在简单图像分类中取得了较好的成绩，但是在更复杂的任务中它存在较大的性能瓶颈。2012 年，亚历克斯·克里切夫斯基等提出的 AlexNet 网络在 ImageNet 大赛上以远超第二名的成绩夺冠，卷积神经网络乃至深度学习引起了学术界广泛的关注，而后发展出的代表性卷积神经网络有 VGG 系列、GoogLeNet 系列、ResNet 系列。

AlexNet 在 LeNet5 网络模型的基础上加深了网络的结构，以学习更丰富的高维的图像特征。AlexNet 网络模型架构如图 8-9 所示。

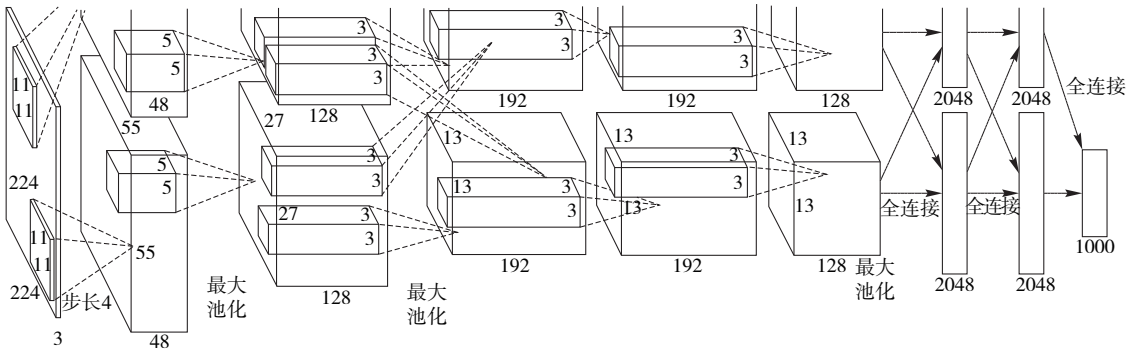


图 8-9　AlexNet 网络模型架构

AlexNet 的网络结构一共包括 5 个卷积层、3 个最大池化层、2 个全连接层，其输入为 3×224×224 大小的 3 通道彩色图像。下面详细介绍 AlexNet 网络模型架构(使用 2 块 GPU)。

输入层：输入为 3×224×224 大小的图像。

卷积层(Conv1)：96 个 11×11 大小的卷积核(每个 GPU 上有 48 个卷积核)，步长为 4。步长是指卷积核在输入图像上移动的步伐大小。

池化层(Pool1)：采用最大池化，窗口大小为 3×3，步长为 2。

卷积层(Conv2)：256 个 5×5 大小的卷积核(每个 GPU 上有 128 个卷积核)。

池化层(Pool2)：采用最大池化，窗口大小为 3×3，步长为 2。

卷积层(Conv3)：384 个 3×3 大小的卷积核(每个 GPU 上有 192 个卷积核)。

卷积层(Conv4)：384 个 3×3 大小的卷积核(每个 GPU 上有 192 个卷积核)。

卷积层(Conv5)：256 个 3×3 大小的卷积核(每个 GPU 上有 128 个卷积核)。

池化层(Pool2)：采用最大池化，窗口大小为 3×3，步长为 2。

全连接层(FC1)：第一个全连接层，将上一层的输出连接成一维向量作为该层的输入，输出 4 096 个神经节点。

全连接层(FC2)：第二个全连接层，输入和输出均为 4 096 个神经节点。

输出层：输出为 1 000 个神经节点，输出的每个神经节点单独对应图像所属类别的概率。在当时的比赛中，使用了 ImageNet 数据集中 1 000 个类别的数据，有 1 000 个分类，因此设定输出的神经节点数为 1 000。

3. VGGNet

AlexNet 网络模型的优越性能启发了业界朝着更深层的神经网络模型方向展开研究。2014 年，IVSVRC14 挑战赛 ImageNet 数据集分类任务的亚军牛津大学 VGG 实验室提出了 VGG11、VGG13、VGG16、VGG19 等一系列网络模型，将网络深度最高提升至 19 层，其架构如图 8-10 所示。

ConvNet Connguration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input(224×224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

图 8-10　VGG 系列网络模型架构

以 VGG16 为例，其输入为 3×224×224 大小的彩色图像，经过 2 个 Conv-Conv-Pooling 单元和 3 个 Conv-Conv-Conv-Pooling 单元的堆叠，最后通过 3 个全连接层输出当前图像分别属于 1 000 个类别的概率分布。

图 8-11 为 VGG16 的网络模型架构，其网络架构一共有 5 个卷积块，每个卷积块都连续多次使用卷积层。该网络一共有 13 个卷积层、5 个池化层、3 个全连接层和 1 个输出层。下面详细介绍 VGG16 网络模型架构。

输入层：输入为 3×224×224 或者 1×224×224 大小的图像。

卷积块(Block1)：连续 2 个卷积层，使用 64 个 3×3 大小的卷积核，输出的特征维度为[224，224，64]。后接 1 个池化层，输出的特征维度为[112，112，64]。

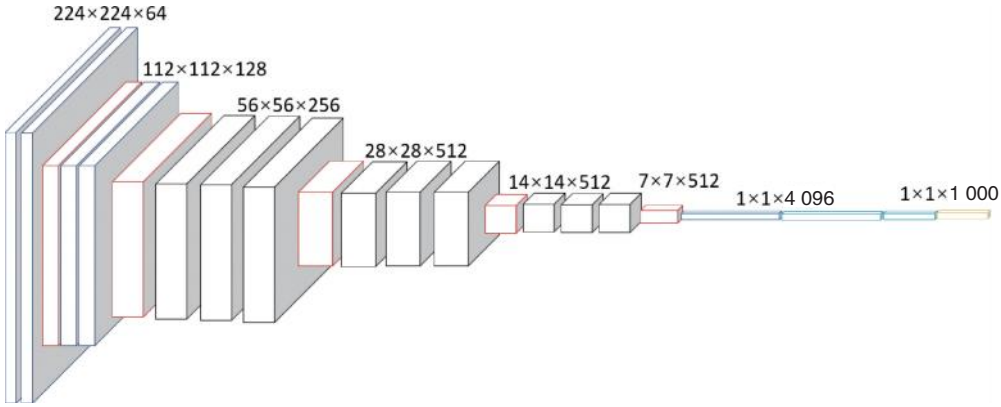


图 8-11　VGG16 网络模型架构

卷积块(Block2)：连续 2 个卷积层，使用 128 个 3×3 大小的卷积核，输出的特征维度为[112，112，128]。后接 1 个池化层，输出的特征维度为[56，56，128]。

卷积块(Block3)：连续 3 个卷积层，使用 256 个 3×3 大小的卷积核，输出的特征维度为[56，56，256]。后接 1 个池化层，输出的特征维度为[28，28，256]。

卷积块(Block4)：连续 3 个卷积层，使用 512 个 3×3 大小的卷积核，输出的特征维度为[28，28，512]。后接 1 个池化层，输出的特征维度为[14，14，512]。

卷积块(Block5)：连续 3 个卷积层，使用 512 个 3×3 大小的卷积核，输出的特征维度为[14，14，512]。后接 1 个池化层，输出的特征维度为[7，7，512]。

全连接层(FC1)：对于上一层输出的特征，向量化为 25 088(7×7×512)个神经元，将其与 4 096 个神经元进行全连接。

全连接层(FC2)：FC1 输出的 4 096 个神经元与 4 096 个神经元进行全连接。

输出层(Softmax)：FC2 输出的 4 096 个神经元与 1 000 个神经元进行全连接，并用 Softmax 函数进行分类预测。

VGG 系列网络模型有以下创新。

(1)网络层数提升至 19 层。

(2)全部采用更小的 3×3 的卷积核，相对于 AlexNet 网络模型中 11×11 的卷积核，参数量更少，计算代价更低。

(3)相比 AlexNet 网络模型中 3×3 的池化窗口，VGG 网络模型采用了更小的 2×2 池化窗口。

4. GoogLeNet

由于 3×3 的卷积核的参数量更少，计算代价更低，同时在性能表现上甚至更优越，因此业界开始探索卷积核最小的情况：1×1 的卷积核。1×1 的卷积核的一个特别之处在于，它可以不改变特征图的宽和高，而只对通道数进行变换。2014 年的 IVSVRC14 挑战赛的冠军是谷歌团队提出的 GoogLeNet 网络模型，该模型大量采用 3×3 和 1×1 的卷积核，它的网络层数达到了 22 层。虽然 GoogLeNet 的层数远大于 AlexNet，但是它的参数量却只有 AlexNet 的 1/12，同时性能也远优于 AlexNet。GoogLeNet 网络模型采用模块化设计的思

想，通过大量堆叠 Inception 模块，形成了复杂的网络结构。图 8-12 是 Inception 模块架构，Inception 模块通过 4 个子网络得到 4 个网络输出，在通道轴上进行拼接合并，形成 Inception 模块的输出。

从图 8-12 可以看出，在 3×3 和 5×5 的卷积核的前面和最大池化的后面都分别加上了 1×1 的卷积核，把得到的特征图与单独使用 1×1 的卷积核得到的特征图联合输出，形成一组新的特征图。1×1 的卷积核可以看作在特征图之间建立的一种全连接，从而得到新的特征图。

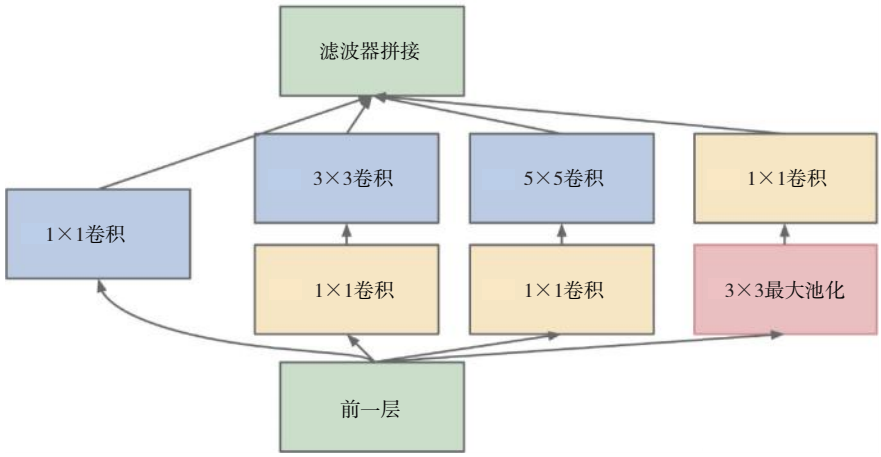


图 8-12　Inception 模块架构

GoogLeNet 网络模型主要由 Inception 模块堆叠而成，其中还包含一些基本的卷积和池化等操作，其架构如图 8-13 所示。

5. ResNet

AlexNet、VGGNet、GoogLeNet 等网络模型的出现将神经网络的发展推向了更深层的阶段，研究人员发现随着网络层数的增加，神经网络的泛化能力有望得到显著提升。然而，随着模型的加深，由于梯度消失和梯度爆炸，网络的训练变得更加困难。特别是在网络层数较深的情况下，这种现象可能更加严重。为了解决深层神经网络中梯度消失和梯度爆炸的问题，2015 年，微软亚洲研究院的何恺明等提出了基于跳跃连接的深度残差神经网络（residual neural network，ResNet），并推出了一系列不同深度的模型，包括 ResNet18、ResNet34、ResNet50、ResNet101 和 ResNet152。这些模型成功地训练出了极深层的卷积神经网络，甚至达到了 1 202 层的网络深度。

ResNet 通过在卷积层的输入和输出之间添加跳跃连接实现层数回退机制，如图 8-14 所示。输入 x 通过两个卷积层，得到特征变换后的输出 $F(x)$，与输入 x 进行对应元素的相加运算，得到最终输出为

$$H(x)=x+F(x)$$

$H(x)$称为残差模块（residual block，ResBlock）。由于被跳跃连接包围的卷积神经网络需要学习映射 $F(x)=H(x)-x$，故称为残差模块。

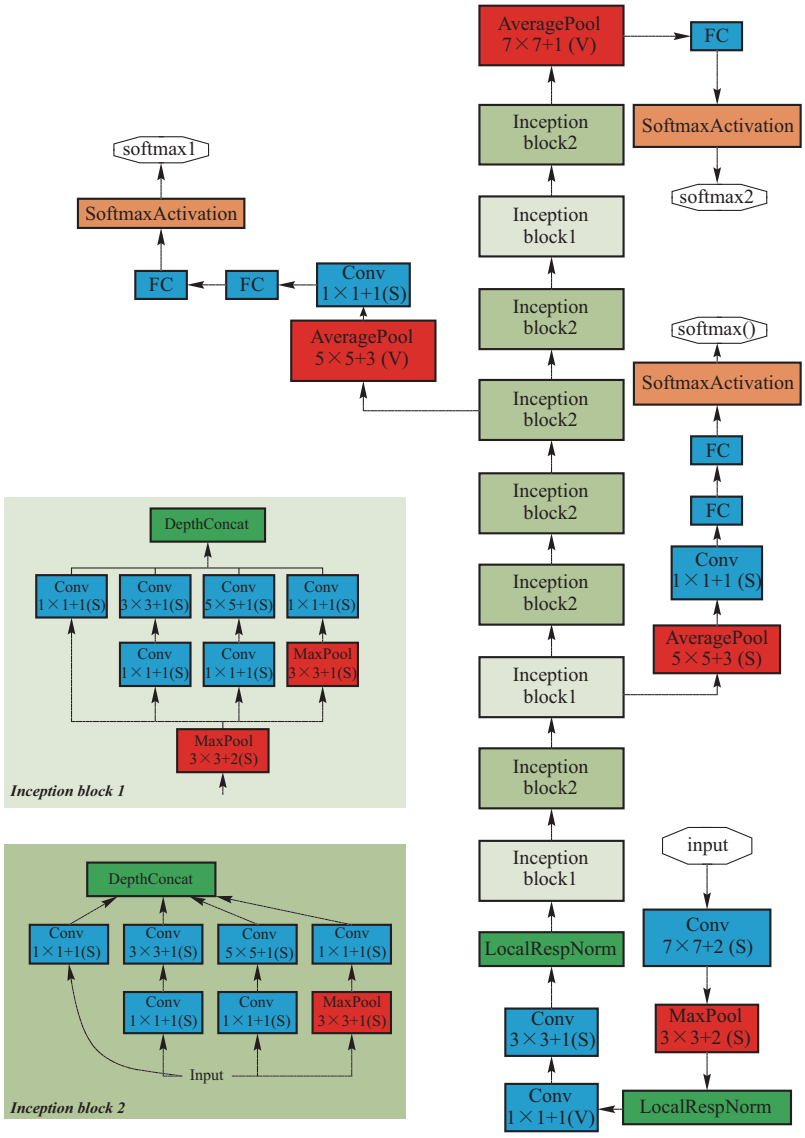


图 8-13　GoogLeNet 网络模型架构

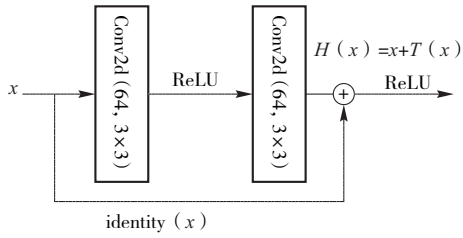


图 8-14　添加跳跃连接

为了使输入 x 与卷积层的输出 $F(x)$能够进行相加运算，需要输入 x 的形状与 $F(x)$的形状完全一致。当形状不一致时，一般通过在跳跃连接上添加额外的卷积运算环节将输入

x 变换到与 $F(x)$ 相同的形状，即 identity(x)，通常使用 1×1 大小的卷积核，主要用于调整输入的形状。

图 8-15 展示了 ResNet50 网络模型架构，可以看到，深度残差神经网络结构通过堆叠残差模块达到了较深的网络层数，从而获得了训练稳定、性能优越的深层网络模型。

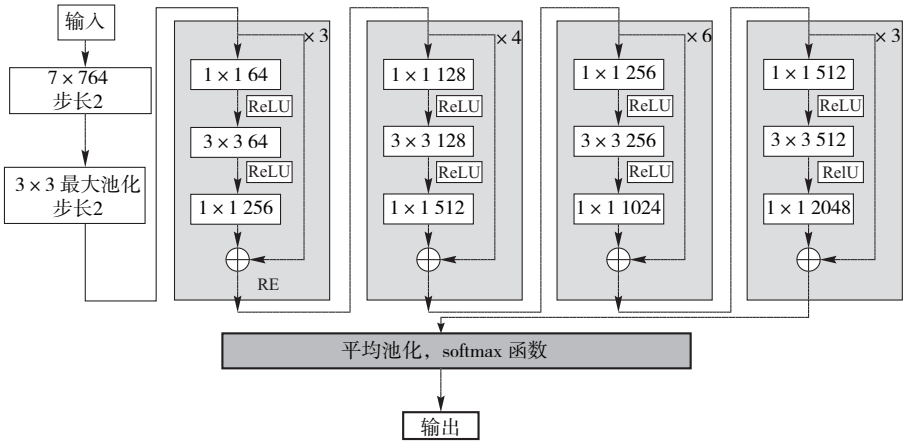


图 8-15　ResNet50 网络模型架构

传统的卷积神经网络或者全连接网络在信息传递的时候或多或少会存在信息丢失、损耗等问题，同时会导致梯度消失或者梯度爆炸等问题，造成网络无法训练。ResNet 网络模型在一定程度上解决了这个问题，通过直接将输入信息跳跃连接到输出，可以保护信息的完整性，整个网络只需要学习输入与输出有差别的那一部分，简化了学习目标和难度。

8.2.3　基于 LeNet5 的 MNIST 手写数字识别

MNIST 数据集是一个广泛应用于手写数字识别任务的数据集，其中包含 0～9 的手写数字样本。该数据集由 4 个部分组成，分别是训练图片集、训练标签集、测试图片集和测试标签集。训练集中包含 60 000 个样本，测试集中有 10 000 个样本。每张图片都是由 28×28 大小的灰度图像构成的。为了方便数据的存储和下载，官方对 MNIST 数据集中的图片进行了处理，将每张图片转换成一个具有 784 个元素的向量，表示为(1，784)的形式。这种处理方式使得数据集更加便于处理和传输。

本节将使用 PyTorch 搭建一个类似于 LeNet5 的卷积神经网络，并使用 MNIST 数据集进行训练，实现手写体数字识别。

(1)导入必要的库，代码如下：

```
import torch
from torch import nn
import torch.nn.functional as F
```

(2)搭建卷积层 C1 和池化层 S2，代码如下：

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        # 输入:(6×28×28); 输出:(6×14×14)
        self.conv1=nn.Sequential(
          nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2),
          nn.ReLU(),
          # 可以选择最大池化 MaxPool2d, 也可以选择平均池化 AvgPool2d, 两者参数相同
          nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
          # input_size=(6×28×28), output_size=(6×14×14)
        )
```

这段代码对应第一个卷积层和第一个池化层。nn.Conv2d( )表示新建一个二维卷积层，可以设置如下参数：第一个参数 in_channels 定义输入通道的数量，第二个参数 out_channels 定义输出通道的数量，kernel_size 定义卷积核的尺寸，stride 定义卷积步长，padding 定义增加的边框尺寸。若输入尺寸为 input_size，则计算(input_size－kernel_size＋2×padding＋stride)/stride，将结果向下取整，即为输出尺寸 output_size。

卷积层后接一个激活函数，后面再接一个最大池化层，这样两层神经网络就搭建完成了。

(3)搭建卷积层 C3 和池化层 S4，代码如下：

```
        # 输入:(6×14×14); 输出: (16×5×5)
        self.conv2=nn.Sequential(
            nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
            # input_size=(6×14×14), output_size=16×10×10
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
            # input_size=(16×10×10), output_size=(16×5×5)
        )
```

这段代码对应的是第二个卷积层和第二个池化层。

(4)搭建全连接层 F5、全连接层 F6 及输出层，代码如下：

```
        # 全连接层
        self.fc1=nn.Sequential(
            nn.Linear(16×5×5, 120),
            nn.ReLU()
        )
        # 全连接层
        self.fc2=nn.Sequential(
            nn.Linear(120, 84),
```

```
            nn.ReLU()
        )
        # 全连接层
        self.fc3=nn.Linear(84, 10)
```

(5)设置网络前向传播，代码如下：

```
    # 网络前向传播过程
    def forward(self, x):
        x=self.conv1(x)
        x=self.conv2(x)
        x=x.view(x.size(0), - 1)
        # 全连接层均使用nn.Linear()线性结构，输入、输出维度均为一维，故需要把数据变换为一维
        x=self.fc1(x)
        x=self.fc2(x)
        x=self.fc3(x)
        return x
```

(6)查看网络结构，代码如下：

```
# 查看网络
myNet=LeNet()
print(myNet)
```

输出的网络结构如图 8-16 所示。

```
LeNet(
  (conv1): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (conv2): Sequential(
    (0): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Sequential(
    (0): Linear(in_features=400, out_features=120, bias=True)
    (1): ReLU()
  )
  (fc2): Sequential(
    (0): Linear(in_features=120, out_features=84, bias=True)
    (1): ReLU()
  )
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

图 8-16　输出的网络结构

(7)载入 MNIST 手写数字识别的训练集和测试集，代码如下：

```
# load data
import torchvision
transform=torchvision.transforms.ToTensor()
# 定义数据预处理方式：转换成 torch.FloatTensor
train_data=torchvision.datasets.MNIST(root='./data',      # 数据存放路径
                                      train=True,         # 是不是训练集
                                      transform=transform,
                                      download=True)      # 是否下载
test_data=torchvision.datasets.MNIST(root='./data',
                                     train=False,
                                     transform=transform,
                                     download=True)
# 数据加载器:组合数据集和采样器
train_loader=torch.utils.data.DataLoader(dataset=train_data, batch_size=64,
shuffle=True)
test_loader=torch.utils.data.DataLoader(dataset=test_data, batch_size=64,
shuffle=False)
```

注意，第一次训练时，需要把参数 download 设置为 True，先下载再训练。

(8)观察部分 MNIST 数据集，代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
def imshow(img):
    img=img/2+0.5
    npimg=img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()
# torchvision.utils.make_grid 将图片进行拼接
imshow(torchvision.utils.make_grid(iter(train_loader).__next__()[0]))
```

输出的 MNIST 数据集图片示例如图 8-17 所示。

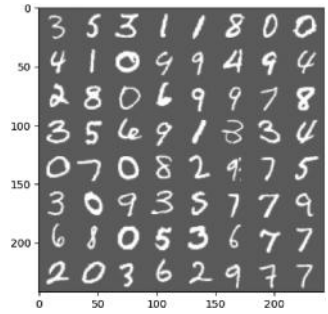


图 8-17　MNIST 数据集图片示例

(9)设置损失函数和优化器，代码如下：

```
# define loss
device=torch.device("cuda" if torch.cuda.is_available( ) else "cpu")
# 若检测到 GPU 环境则使用 GPU，否则使用 CPU
net=LeNet( ).to(device)     # 实例化网络，有 GPU 则将网络放入 GPU 加速
loss_fuc=nn.CrossEntropyLoss( )     # 多类分类问题，选择交叉熵损失函数
optimizer=torch.optim.SGD(net.parameters( ), lr=0.001, momentum=0.9)
# 选择 SGD 算法，学习率取 0.001
```

(10)训练模型，代码如下：

```
# 开始训练
EPOCH=8  # 迭代次数
loss_list=[ ]
for epoch in range(EPOCH):
    sum_loss=0
    # 数据读取
     for i, data in enumerate(train_loader):
        inputs, labels=data
        inputs, labels=inputs.to(device), labels.to(device)
        # 有 GPU 则将数据放入 GPU 加速
        # 梯度清零
        optimizer.zero_grad( )
        # 传递损失+更新参数
        output=net(inputs)
        loss=loss_fuc(output, labels)
        loss.backward( )
        optimizer.step( )
        # 每训练 100 个 batch 打印一次平均损失
        sum_loss+=loss.item( )
        if i%100==99:
            print('[Epoch:% d, batch:% d] train loss:% .03f'% (epoch+1, i+1, sum_loss/100))
            loss_list.append(sum_loss/100)
            sum_loss=0.0
    correct=0
    total=0
    for data in test_loader:
        test_inputs, labels=data
        test_inputs, labels=test_inputs.to(device), labels.to(device)
        outputs_test=net(test_inputs)
        _, predicted=torch.max(outputs_test.data, 1)  # 输出得分最高的类
```

```
            total+=labels.size(0)  # 统计图片的总个数
            correct+=(predicted==labels).sum()  # 统计正确分类的个数
            print('第{}个 epoch 的识别准确率为:{}% '.format(epoch+1, 100* correct.item()
/total))
    save_path='./Lenet.pth'
    torch.save(net.state_dict(), save_path)
```

LeNet5 模型在 MNIST 数据集上的训练过程如图 8-18 所示。

```
[Epoch:8, batch:100] train loss: 0.084
[Epoch:8, batch:200] train loss: 0.096
[Epoch:8, batch:300] train loss: 0.092
[Epoch:8, batch:400] train loss: 0.086
[Epoch:8, batch:500] train loss: 0.078
[Epoch:8, batch:600] train loss: 0.094
[Epoch:8, batch:700] train loss: 0.086
[Epoch:8, batch:800] train loss: 0.078
[Epoch:8, batch:900] train loss: 0.080
第8个epoch的识别准确率为：97.52%
```

图 8-18　LeNet5 模型在 MNIST 数据集上的训练过程

中间隐藏了部分迭代结果，网络的训练集为 60 000 张图片，验证集为 10 000 张图片，EPOCH＝8，识别准确率为 97.52%。

(11)打印训练过程中的损失值变化曲线，代码如下：

```
    # 打印损失值变化曲线
    x=[ ]
    for i in range(len(loss_list)):
        x.append(i)
    print(x)
    import matplotlib.pyplot as plt
    plt.plot(x, loss_list)
    plt.title('traning loss')
    plt.xlabel('epochs')
    plt.ylabel('loss')
    plt.show()
```

训练过程中的损失值变化曲线如图 8-19 所示，随着迭代轮数的增加，训练的损失值先急剧下降，后趋于振荡稳定，损失值最终控制在 0.1 左右。

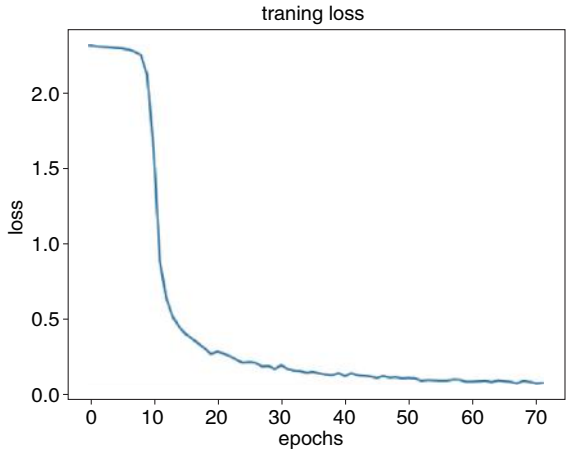


图 8-19　训练过程中损失值变化曲线

8.3 循环神经网络

循环神经网络(recurrent neural network，RNN)以序列数据作为输入，且在序列的演进方向进行递归，所有节点(循环单元)按链式连接进行递归。

使用 PyTorch 搭建循环神经网络

从生物神经学角度看 RNN，可以认为其是对生物神经系统交互式联系(recurrent connection)的简单模拟，这种交互式联系在大脑新皮质(neocortex)中是普遍存在的。这也从侧面反映了人类学习是一个动态变化的过程，因而对该神经元的模拟在生物工程上有着重要的意义。

RNN 是深度学习的一个重要分支，近年来与 RNN 模型相关的研究发展迅速。RNN 的应用场景有手写字体识别、语音识别、自然语言处理和基于计算机视觉等序列问题。

RNN 主要处理序列数据，其最大特点就是神经元在某时刻的输出可以再次输入到神经元，这种串联的网络结构非常适合时间序列数据，它可以保持数据中的依赖关系。到目前为止，具有代表性的 RNN 模型有长短期记忆网络及门控循环单元(gated recurrent unit，GRU)模型。

到目前为止，只是初步介绍了循环神经网络的基本概念，下面通过深入分析来全方位介绍循环神经网络。

8.3.1　标准循环神经网络

一个标准的 RNN 模型主要由输入层、隐藏层和输出层组成。传统神经网络的结构都是从输入层到隐藏层再到输出层，层与层之间是全连接或部分连接的，但每层内部的节点是无连接的，而 RNN 隐藏层内部的节点是有连接的，隐藏层的输入不仅包括输入层的输出，还包括上一时刻隐藏层得到的输出。这种位于同一层的神经元节点之间的连接，最终使 RNN 实现了时间记忆功能。最简单的 RNN 模型结构如图 8-20 所示。

图 8-20(a)为 RNN 结构图的折叠形式，图 8-20(b)为 RNN 结构图的展开形式，从图中

可以看出，隐藏层的输出有两个：一个输出反馈给自己，一个输出到下一时刻的神经元中。在每一时刻，模块 $\boldsymbol{S}$ 在读取了当前时刻的输入 $\boldsymbol{x}_t$ 和上一时刻的隐藏状态 $\boldsymbol{s}_{t-1}$ 之后会产生新的隐藏状态 $\boldsymbol{s}_{t+1}$，并产生本时刻的输出 $\boldsymbol{h}_t$。由于模块 $\boldsymbol{S}$ 中的运算和参数在不同时刻是相同的，因此可以将模块 $\boldsymbol{S}$ 看作同一神经网络结构被无限复制的结果。正如 CNN 在不同空间位置共享参数，RNN 在不同时间共享参数，从而能够使用有限的参数处理任意长度的序列。下面对 RNN 模型进行数学描述。

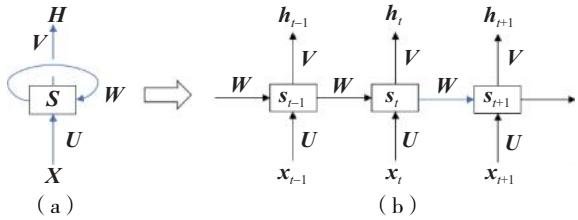


图 8-20　最简单的 RNN 模型结构

$\boldsymbol{x}_t$：t 时刻的输入，$\boldsymbol{X}=[\boldsymbol{x}_1, \cdots, \boldsymbol{x}_{t-1}, \boldsymbol{x}_t, \boldsymbol{x}_{t+1}, \cdots, \boldsymbol{x}_T]$为输入序列。

$\boldsymbol{s}_t$：t 时刻的隐藏状态，也称为网络的记忆单元，一个隐藏状态可以包含多个神经元，$\boldsymbol{S}=[\boldsymbol{s}_1, \cdots, \boldsymbol{s}_{t-1}, \boldsymbol{s}_t, \boldsymbol{s}_{t+1}, \cdots, \boldsymbol{s}_T]$为隐藏状态序列。

$\boldsymbol{h}_t$：t 时刻的输出，$\boldsymbol{H}=[\boldsymbol{h}_1, \cdots, \boldsymbol{h}_{t-1}, \boldsymbol{h}_t, \boldsymbol{h}_{t+1}, \cdots, \boldsymbol{h}_T]$为输出序列。

$\boldsymbol{U}$：输入序列信息 $\boldsymbol{X}$ 的权重参数矩阵。

$\boldsymbol{W}$：隐藏状态 $\boldsymbol{S}$ 的权重参数矩阵。

$\boldsymbol{V}$：输出序列信息 $\boldsymbol{H}$ 的权重参数矩阵。

$\boldsymbol{s}_t$ 是根据上一个时刻 $t-1$ 的隐藏状态 $\boldsymbol{s}_{t-1}$ 和当前时刻 t 的输入 $\boldsymbol{x}_t$ 计算得到的。假设函数 f 为隐藏状态的激活函数，当前隐藏状态 $\boldsymbol{s}_t$ 的计算公式为

$$\boldsymbol{s}_t=f(\boldsymbol{W}\boldsymbol{s}_{t-1}+\boldsymbol{U}\boldsymbol{x}_t) \tag{8-1}$$

假设函数 g 为输出的激活函数，则 t 时刻输出的计算公式为

$$\boldsymbol{h}_t=g(\boldsymbol{V}\boldsymbol{s}_t) \tag{8-2}$$

将式(8-2)反复代入式(8-1)，可得

$$\begin{aligned}\boldsymbol{h}_t &= g(\boldsymbol{V}\boldsymbol{s}_t)=g(\boldsymbol{V}f(\boldsymbol{W}\boldsymbol{s}_{t-1}+\boldsymbol{U}\boldsymbol{x}_t))=g(\boldsymbol{V}f(\boldsymbol{W}f(\boldsymbol{W}\boldsymbol{s}_{t-2}+\boldsymbol{U}\boldsymbol{x}_{t-1})+\boldsymbol{U}\boldsymbol{x}_t))\\ &=g(\boldsymbol{V}f(\boldsymbol{W}f(\boldsymbol{W}f(\boldsymbol{W}\boldsymbol{s}_{t-3}+\boldsymbol{U}\boldsymbol{x}_{t-2})+\boldsymbol{U}\boldsymbol{x}_{t-1})+\boldsymbol{U}\boldsymbol{x}_t))=\cdots\end{aligned}$$

RNN 模型由式(8-1)和式(8-2)组成，与人工神经网络模型相比，RNN 模型的参数多了 $\boldsymbol{W}$ 和 $\boldsymbol{V}$ 两个权重参数矩阵，并且网络数据传递方式也发生了改变。值得注意的是，图 8-20 的 RNN 模型的基本结构中每一个时刻 t 都会对应一个输出 $\boldsymbol{h}_t$，但在实际情况中，每一个时刻是否会有对应的输出，需要根据具体的任务需求变化。

RNN 模型并不是只有如图 8-20 所示的一种结构，其网络结构经过排列组合可以有多种变化。图 8-21 为单层 RNN 模型的各种结构，图中矩形框代表隐藏状态，箭头表示数据流动方向。

图 8-21(a)为一对一结构，单输入，单输出。该 RNN 模型适用于词性分类、时序回归或者图像分类问题，如输出单词的词性。

图 8-21(b)为一对多结构，单输入，序列输出。该 RNN 模型适用于图片字幕预测，如输入一张图片后输出一段文字序列；也可以作为解码器，如先训练好网络中的权重参数，用一个单词解码一个句子。

图 8-21(c)为多对一结构，序列输入，单输出。该 RNN 模型适用于文本分类、视频分类等。

图 8-21(d)为多对多结构，序列输入，序列输出，输入与输出不同步。该 RNN 模型适用于机器翻译，如读入英文语句然后将其以语法形式输出。

图 8-21(e)为多对多结构，序列输入，序列输出，输入与输出同步。该 RNN 模型适用于机器翻译模型、视频翻译字幕工具、机器问答系统等众多场合。

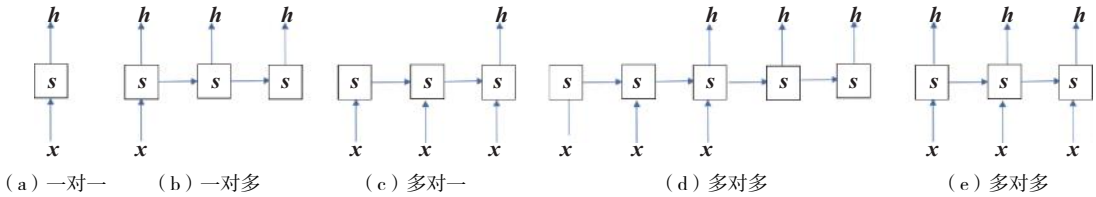


图 8-21　单层 RNN 模型的各种结构

单向 RNN 只能用到前面的信息，而不能利用后面的信息。鉴于单向 RNN 在需要关联未来数据等情况下存在不足，人们提出了双向 RNN。双向 RNN 同时考虑后面的信息，用未来的信息来帮助前面的信息，从而做出决定。

深度循环神经网络是在基本 RNN 结构的基础上改进得到的，为了增强模型的表达能力，可以在网络中设置多个循环层，每一个时刻 t 对应多个隐藏状态，将每层循环网络中的输出传给下一层处理，因而形成深度循环神经网络。该模型结构能够带来更好的学习能力，缺点在于难以对网络进行控制，并且随着网络层数的增多会带来梯度消失或梯度爆炸等问题。因此需要根据实际问题确定选择哪一种 RNN 结构。

8.3.2　长短期记忆网络

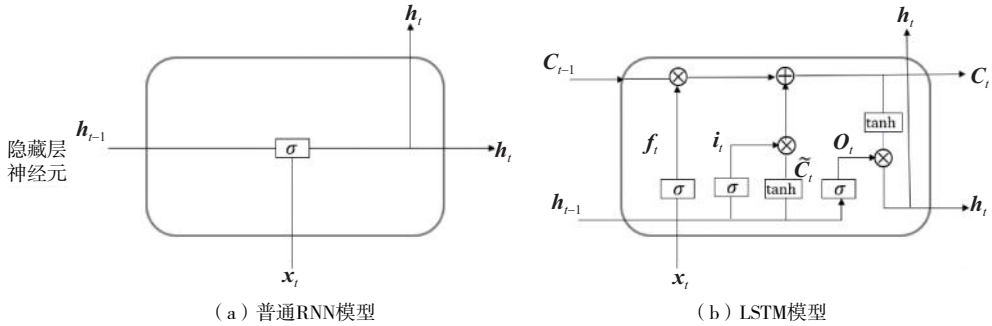
虽然 RNN 模型在设计之初就是为了解决序列数据的长期依赖性，但大量实践表明，标准的 RNN 由于存在梯度消失和梯度爆炸等问题，往往难以实现信息的长期保存。为解决长期依赖、梯度消失和梯度爆炸的问题，泽普·霍赫赖特(Sepp Hochreiter)提出 LSTM 模型，用于改进传统的 RNN 模型。

LSTM 模型在序列数据的实际应用中效果更好，相较于 RNN 模型的隐藏单元，LSTM 模型的隐藏单元的内部结构更加复杂，在信息沿着网络流动的过程中，LSTM 模型能够通过增加线性干预有选择地添加或者减少信息，从而解决了普通 RNN 模型记不住的问题(在处理长序列数据时面临的长期依赖问题及梯度消失和梯度爆炸问题)。

LSTM 模型与传统的 RNN 模型在时序的传播方式上并没有本质的差异，区别在于它们使用不同的方式计算隐藏层神经元的状态。LSTM 模型中的记忆功能被称为 Cells，这些 Cells 在神经元内部决定是否应该写入或删除对信息的记忆，并且可以将之前的状态、现在的记忆和当前输入的信息结合在一起，对长期信息进行记录。经过十几年的发展，LSTM

模型被证明比传统的 RNN 模型在处理长期依赖问题上更加有效，LSTM 模型自然而然地成为循环神经网络中最经典的网络模型。下面重点介绍 LSTM 模型的网络结构及其特性。

普通 RNN 模型的结构如图 8-22(a)所示，LSTM 模型为 RNN 模型的变体，也具有这种类似的链式结构，但 LSTM 模型的重复模块 $\boldsymbol{S}$ 与一般 RNN 模型相比具有不同的结构。LSTM 模型较传统 RNN 模型而言，模块 $\boldsymbol{S}$ 中的神经网络层是 4 个而不是 1 个，并且这 4 个神经网络层以一种特殊的方式进行交互。图 8-22(b)为常见的 LSTM 模型结构。



(a) 普通RNN模型　　(b) LSTM模型

图 8-22　普通 RNN 模型和 LSTM 模型的结构

图 8-23 是对图 8-22(b)中部分结构的解释。圆圈表示逐点计算，如向量加法；方框代表学习的神经网络层，行合并表示串联，而分支表示内容正在被复制，并且副本将转移到不同的位置。

图 8-23　LSTM 模型结构中部分结构的解释

下面将 LSTM 模型中的记忆功能(Cells)统一称为记忆单元。记忆单元由 4 个要素组成，包括输入门、遗忘门、输出门及自循环连接神经元，如图 8-24 所示。其中，门机制将输出范围限制为 0～1 之间的数值，用于描述每个组成部分通过的信息量。数值 0 表示不允许任何信息通过，而数值 1 表示允许所有信息通过。LSTM 是最早提出的循环神经网络门控算法，它利用输入门、遗忘门和输出门这三个门来维护和控制记忆状态。

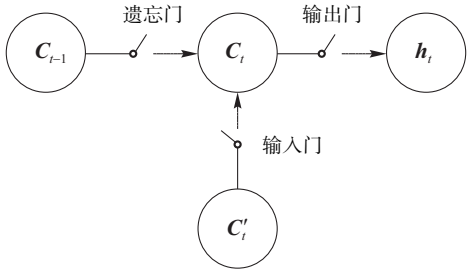


图 8-24　LSTM 信息流的门限控制

图 8-24 中，$\boldsymbol{C}_t$ 表示记忆功能信息($\boldsymbol{C}_{t-1}$ 表示前一时刻的常时态信息)，$\boldsymbol{C}_t'$表示短时态信

息，$\boldsymbol{h}_t$ 为经过 LSTM 单元后的输出信息，3 条线上的开关为门限开关。

LSTM 的第一步是确定从记忆功能中舍弃哪些信息。这由遗忘门决定，如图 8-25 所示。LSTM 接收 $\boldsymbol{h}_{t-1}$ 和 $\boldsymbol{x}_t$，并且对具有记忆功能的 $\boldsymbol{C}_{t-1}$ 中的每一个数来说，LSTM 的输出值都介于 0～1 之间，1 表示完全接受，0 表示完全忽略，用数学表达式描述为

$$\boldsymbol{f}_t=\sigma(\boldsymbol{W}_f\cdot[\boldsymbol{h}_{t-1},\ \boldsymbol{x}_t]+\boldsymbol{b}_f)$$

其中，$\boldsymbol{f}_t$ 为 t 时刻遗忘门的状态，$\boldsymbol{W}_f$ 为遗忘门的权重矩阵，$\boldsymbol{b}_f$ 为遗忘门的偏置，$\boldsymbol{x}_t$ 为当前时刻的外部输入，$\boldsymbol{h}_{t-1}$ 为上一时刻的输出，σ 为门限激活(控制)函数。

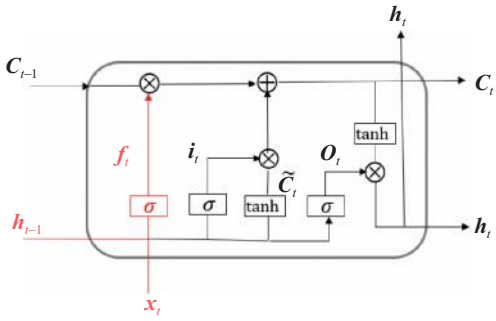


图 8-25　需遗忘的信息

LSTM 的第二步是确定需要在记忆功能中保存哪些新信息，如图 8-26 所示。这一步的工作分成两部分完成。

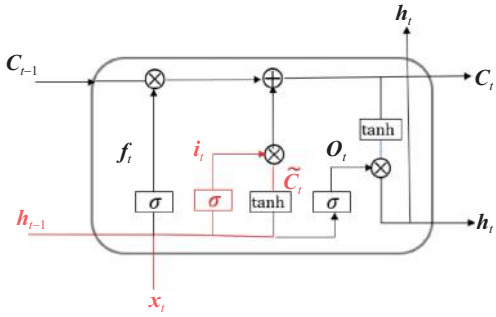


图 8-26　需保存的信息

第一部分，用一个输入门的网络层确定哪些信息需要更新。当前时刻(t 时刻)的输入门状态 $\boldsymbol{i}_t$ 用数学表达式描述为

$$\boldsymbol{i}_t=\sigma(\boldsymbol{W}_i\cdot[\boldsymbol{h}_{t-1},\ \boldsymbol{x}_t])+\boldsymbol{b}_i$$

其中，$\boldsymbol{W}_i$ 为输入门的权重参数矩阵，$\boldsymbol{b}_i$ 为偏置。

第二部分，在 tanh 网络层创建一个新的备选值向量，即$\tilde{\boldsymbol{C}}_t$，用来表示添加到记忆功能状态的信息，即当前时刻的输入信息，包括前一时刻的输出 $\boldsymbol{h}_{t-1}$ 及当前的外部输入 $\boldsymbol{x}_t$。当前时刻的输入信息可以表示为

$$\tilde{\boldsymbol{C}}_t=\tanh(\boldsymbol{W}_C\cdot[\boldsymbol{h}_{t-1},\ \boldsymbol{x}_t]+\boldsymbol{b}_C)$$

其中，$\boldsymbol{W}_C$ 为权重参数矩阵，$\boldsymbol{b}_C$ 为偏置，$\tilde{\boldsymbol{C}}_t$ 为添加到记忆功能状态的信息。

LSTM 的第三步为记忆功能的更新，即将旧的记忆功能状态 $\boldsymbol{C}_{t-1}$ 更新到新的记忆功能状态 $\boldsymbol{C}_t$，如图 8-27 所示。这里将旧的状态 $\boldsymbol{C}_{t-1}$ 乘以遗忘门的状态 $\boldsymbol{f}_t$，用来去除需要忘记的信息，从而留下有用的信息；然后加上 $\boldsymbol{i}_t$ 和 $\tilde{\boldsymbol{C}}_t$ 的乘积，即可完成记忆功能状态的更新，表达式为

$$\boldsymbol{C}_t=\boldsymbol{f}_t\otimes\boldsymbol{C}_{t-1}+\boldsymbol{i}_t\otimes\widetilde{\boldsymbol{C}}_t$$

其中，$\tilde{\boldsymbol{C}}_t$ 表示用来添加到记忆功能的信息，$\otimes$表示矩阵对应元素相乘。

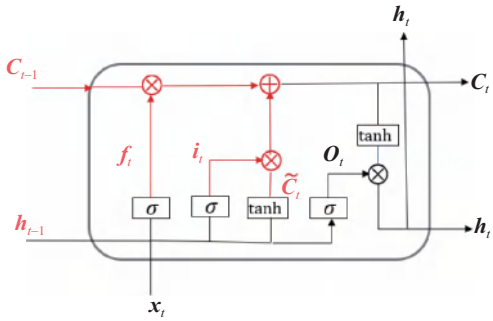


图 8-27　记忆功能状态更新

LSTM 的最后一步为确定输出值，如图 8-28 所示。输出依赖记忆功能状态，但会是一个过滤的版本。首先，运行输出门的网络层，用来确定记忆状态中哪些部分可以输出；然后，把记忆状态输入的 tanh 值(把数值调整到－1～1 之间)和网络层的输出值相乘，从而输出想要输出的部分。整个过程的数学表达式为

$$\boldsymbol{O}_t=\sigma(\boldsymbol{W}_O\cdot[\boldsymbol{h}_{t-1},\ \boldsymbol{x}_t]+\boldsymbol{b}_O)$$

$$\boldsymbol{h}_t=\boldsymbol{O}_t\otimes\tanh(\boldsymbol{C}_t)$$

其中，$\boldsymbol{O}_t$ 表示输出门状态，$\boldsymbol{W}_O$ 为权重参数矩阵，$\boldsymbol{b}_O$ 为偏置，$\boldsymbol{h}_t$ 为 t 时刻的输出，$\boldsymbol{C}_t$ 为当前时刻新的记忆状态。

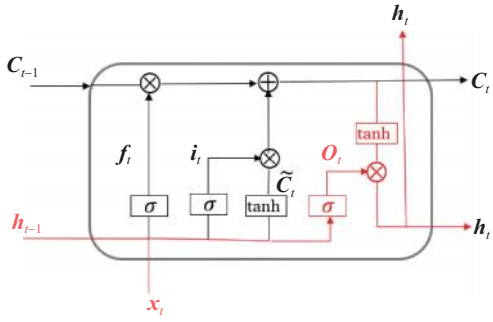


图 8-28　LSTM 的输出

LSTM 与 RNN 在工作方式上基本相似，其不同之处在于 LSTM 引入了一种更精细的内部处理单元，以实现有效的上下文信息存储和更新。通过这种更加细化的机制，LSTM 能够更好地捕捉和利用序列数据中的长期依赖关系。由于其出色的性能，LSTM 已广泛应

用于与序列学习相关的众多任务中，例如语音识别、语言模型、词性标注、机器翻译等。谷歌公司已成功地将 LSTM 应用于其智能手机上的语音识别功能及谷歌翻译服务中，以提供更准确和流畅的语音识别与翻译体验。

8.3.3 循环神经网络与自然语言处理

RNN 模型在自然语言处理领域的应用非常广泛，包括问答系统、情感分析、图像到文字的映射、机器翻译、语音识别、词性标注、命名实体识别等。这些应用领域都取得了显著的进展，彰显了 RNN 模型在自然语言处理任务中的价值。下面介绍一些 RNN 模型的具体应用案例，以进一步了解其应用价值。

(1)语音识别。语音识别是指由输入的语音数据生成相应的文本信息。RNN 模型在语音识别中有着重大突破。以微信的语音转文字为例，语音识别从结构上可以分为前端和后端两部分，如图 8-29 所示。前端包括语音获取和预处理；后端包括语音信号的特征提取和 RNN 模型的语音识别，并最终对识别的结果进行对比和评估。

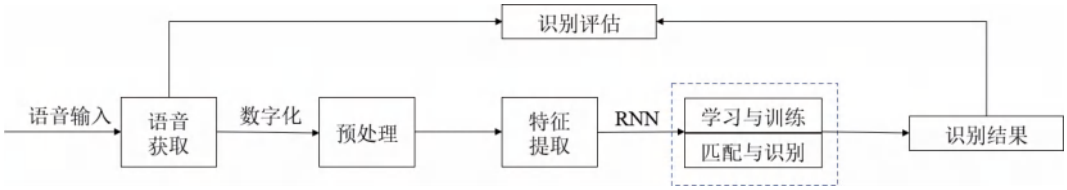


图 8-29　语音识别结构图

(2)机器翻译。机器翻译是指从一种语言变换成另外一种语言。对于计算机来说，翻译就是一个解码再编码的过程。在翻译时，RNN 将源语言作为输入序列，把翻译语言作为输出序列。由于每次输出都会参考上一次输出的结果，所以基于 RNN 模型的机器翻译更具有整体性，而不是简单地翻译单词。

(3)语言建模和文本生成。2016 年，谷歌发布并升级了语言处理框架 SyntaxNet，如图 8-30 所示。

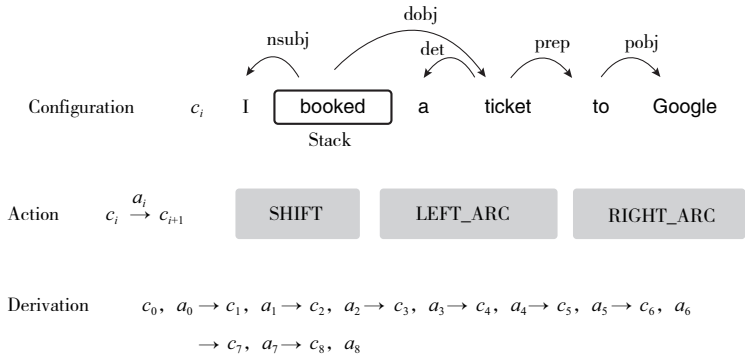


图 8-30　语言处理框架 SyntaxNet

SyntaxNet 是一个用于分析和理解语法结构的 RNN 模型，在自然语言处理的语义理解领域的最高识别率上提高了 25%，为 40 种语言带来了新的文本分割和词态分析功能。

8.4 项目实践——美食“图”鉴

8.4.1　项目介绍

在日常生活或节日盛典中，人们常常会遇到各式各样的食物，而某些人眼中的黑暗料理可能正是另一些人的喜好。然而，有时候人们也会面临无法准确命名食物的尴尬情况。在初步了解深度学习的基础上，是否可以搭建一个神经网络，通过机器自动识别那些“欲言又止”的美食？

答案当然是肯定的。读者可以利用本章所学的理论知识，通过卷积神经网络实现一个美食图片分类项目，即对常见食物图片进行分类以识别食物。要实现这个完整项目，需要理清思路。首先，需要获取数据。数据在深度学习的各个应用领域中起着至关重要的作用。如果没有合适的数据(从数量和质量两方面考虑)，即使拥有再好的算法和网络结构，也无法取得理想结果。这里的数据指的是图片及其对应的类别标签，通常还需要对这些图片进行预处理。然后，可以进入网络搭建过程，构建一个卷积神经网络实现美食图片分类。有了数据和网络结构之后，就需要通过训练网络来学习。在训练过程中，针对不同的食物类别，网络将从中学习到各种细节特征和语义信息，从而能够区分各种不同的食物。网络训练的效果还需要通过“考试”进行验证。这里的“考试”是指将训练好的模型应用于新的食物图片，并检验其分类效果。

基于上述流程分析，接下来将分步骤开始动手实战。本节所设计的代码部分参考了 https://github.com/chouxianyu/LHY_ML2020_Codes 的内容，读者可以到 GitHub 网站学习。

8.4.2　数据准备

根据上述分析可知，首先需要准备大量的食物图片，以及为每张图片打上类别标签，并且按照一定的比例将图片划分成训练集和测试集。食物图片可以通过自己拍摄或者通过爬虫下载，还可以使用现有的公共数据集。本书选用两个不同大小的食物图片数据集，这两个数据集分别包含 11 个类别(记为 Food-11)和 101 个类别(记为 Food-101)的食物，可满足不同需求下的美食“图”鉴任务。

读者可在 Kaggle 网站下载 Food-11 数据集，网址为 https://www.kaggle.com/c/ml2020spring-hw3/data。该数据集包含的 11 类食物分别为面包(bread)、乳制品(dairy product)、甜点(dessert)、鸡蛋(egg)、油炸食品(fried food)、肉类(meat)、面条/意大利面(noodles/pasta)、米饭(rice)、海鲜(seafood)、汤类(soup)、蔬菜/水果(vegetable/fruit)。Food-11 的原始数据集被划分成了训练集(9 866 张图片)、验证集(3 430 张图片)和测试集(3 347 张图片)。Food-101 数据集是一个更为庞大的食物图片数据集，从苹果派到华夫饼，该数据集对各种食物提供了细致的分类，可以在 https://www.kaggle.com/dataset/kmader/food41 下载 Food-101 数据集。Food-101 的原始数据集被划分成了训练集和测试集。

下载好数据集的压缩包之后，需要对原始数据进行一些预处理操作，后续才能将其送入网络中训练。这里以 PyTorch 为深度学习框架，以 Food-11 数据集为例进行说明。

首先，需要导入必要的库。

```
import os
import numpy as np
import cv2
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import pandas as pd
from torch.utils.data import DataLoader, Dataset
import time
```

然后，设计 readimg( )函数，通过 OpenCV 库读取 Food-11 数据集的图片及标签，并存放在数组中。

```
def readimg(path, label):
    # path 为数据集的路径，label 为训练集和验证集需要返回的标签
    image_dir=sorted(os.listdir(path))
    x=np.zeros((len(image_dir), 128, 128, 3), dtype=np.uint8)
    y=np.zeros((len(image_dir)), dtype=np.uint8)
    # 遍历所有图片，图片和标签分别填充在数组 x 和 y 中
    for i, file in enumerate(image_dir):
        img=cv2.imread(os.path.join(path, file))
        x[i,:,:]=cv2.resize(img, (128, 128))  # 大小调整为 128×128
        if label:
            y[i]=int(file.split("_")[0]) # 通过分割符切分得到标签
        if label:
            return x, y
        else:
            return x
```

将 Food-11 的训练集、验证集和测试集通过 readimg( )函数读入，并查看各个子数据集的图片数量，运行后输出训练集的图片数量为 9 866 张，验证集的图片数量为 3 430 张，测试集的图片数量为 3 347 张。

```
workspace_dir='Food-11 文件夹的保存路径/food-11'
print("开始读入数据")
train_x, train_y=readimg(os.path.join(workspace_dir,"training"), True)
print("训练集的数量为:{}".format(len(train_x)))
val_x, val_y=readimg(os.path.join(workspace_dir,"validation"), True)
print("验证集的数量为:{}".format(len(val_x)))
test_x=readimg(os.path.join(workspace_dir, "testing"), False)
print("测试集的数量为:{}".format(len(test_x)))
```

定义数据增强方式和 ImgDataset 类，以便将数据集加载进后续的算法模型中进行训练、测试。

```
# 进行数据增强操作
train_transform=transforms.Compose([
    transforms.ToPILImage( ),
    transforms.RandomHorizontalFlip( ), # 随机将图片进行水平翻转
    transforms.RandomRotation(15), # 随机旋转图片
    transforms.ToTensor( ), # 将图片转成 Tensor 类型
])
# 测试集不需要进行数据增强操作
test_transform=transforms.Compose([
    transforms.ToPILImage( ),
    transforms.ToTensor( ),
])
class ImgDataset(Dataset):
    def__init__(self, x, y=None, transform=None):
        self.x=x
        self.y=y
        if y is not None:
            self.y=torch.LongTensor(y)  # 将 y 转成 Tensor 类型
            self.transform=transform
    def__len__(self):
        return len(self.x)
    def__getitem__(self, index):
        X=self.x[index]
        if self.transform is not None:
            X=self.transform(X)
        if self.y is not None:
            Y=self.y[index]
            return X, Y
        else:
            return X
```

通过定义的 ImgDataset 类和 DataLoader 工具得到可用于网络训练和验证的数据集 train_loader 和 val_loader。

```
batch_size=128
train_set=ImgDataset(train_x, train_y, train_transform)
val_set=ImgDataset(val_x, val_y, test_transform)
train_loader=DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader=DataLoader(val_set, batch_size=batch_size, shuffle=False)
```

8.4.3 网络搭建

本小节的目的是利用卷积神经网络实现食物图片分类任务，卷积神经网络结构的设计至关重要，网络的搭建可简单、可复杂，通过利用普通的卷积、池化、归一化、激活函数、全连接等基本操作进行模型组网，完全可以实现一个食物图片分类器。这里为读者展示一个 PyTorch 版本的简单的食物图片分类器，将其命名为 Fclassifier。

```
class Fclassifier(nn.Module):
    def __init__(self):
        super.__init__()
        self.cnn=nn.Sequential(
            nn.Conv2d(3, 64, 3, 1, 1),
            # 输出的特征维度为[64, 128, 128]，64 个卷积核
            nn.BatchNorm2d(64),   # 批归一化处理
            nn.ReLU(),
            nn.MaxPool2d(2, 2, 0), # 输出的特征维度为[64, 64, 64]
            # 最大池化之后，图片的长和宽减小为原来的一半

            nn.Conv2d(64, 128, 3, 1, 1),
            # 输出的特征维度为[128, 64, 64]，128 个卷积核
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2, 2, 0),   # 输出的特征维度为[128, 32, 32]

            nn.Conv2d(128, 256, 3, 1, 1),
            # 输出的特征维度为[256, 32, 32]，256 个卷积核
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2, 2, 0),   # 输出的特征维度为[256, 16, 16]

            nn.Conv2d(256, 512, 3, 1, 1),
            # 输出的特征维度为[512, 16, 16]，512 个卷积核
            nn.BatchNorm2d(512),
            nn.ReLU(),
            nn.MaxPool2d(2, 2, 0),   # 输出的特征维度为[512, 8, 8]

            nn.Conv2d(512, 512, 3, 1, 1),
            # 输出的特征维度为[512, 8, 8]，512 个卷积核
            nn.BatchNorm2d(512),
            nn.ReLU(),
            nn.MaxPool2d(2, 2, 0),   # 输出的特征维度为[512, 4, 4]
```

```
                # 最后输出的特征将展开为一维，输入全连接网络
        )
    self.fc=nn.Sequential(
          nn.Linear(512* 4* 4, 1024),
          nn.ReLU( ),
          nn.Linear(1024, 512),
          nn.ReLU( ),
          nn.Linear(512, 11)              #11 个类别，11 种输出
        )

    def forward(self, x):
        out=self.cnn(x)
        out=out.view(out.size( )[0], -1)
        return self.fc(out)
```

在上述代码中，nn.Conv2d(in_channels，out_channels，kernel_size，stride，padding)函数的参数分别为输入通道数、输出通道数、卷积核的大小、滑动步长、填充大小；nn.MaxPool2d(kernel_size，stride，padding)函数的参数分别为池化核大小、滑动步长和填充大小。食物图片分类器 Fclassifier 网络的输入大小为 3×128×128，之后经历 5 次卷积－正规化－激活－池化操作，该操作具体包含 3×3 卷积、批归一化处理、ReLU 激活函数、最大池化，每经过一次步长为 2 的最大池化 MaxPool2d，特征图的长和宽都会减小为原来的一半；最后经历 3 个全连接层，输出包含 11 个元素的特征向量，对应美食“图”鉴任务 Food-11 数据集中要识别的 11 个类别。

8.4.4　模型训练与测试

在完成网络模型结构的搭建后，需要让该模型通过 Food-11 数据集的训练集来学习各类食物图片的特征，这个过程称为模型训练，它指导着网络模型的学习过程。同时，还需要及时评估模型训练的效果，即在 Food-11 数据集的验证集上检查经过一定训练轮数后模型的表现如何。在分类任务中，可以使用准确率作为评价指标。模型可以边训练边验证，也可以在所有训练轮数完成后再进行验证。前者可以类比为“阶段测验”，后者类似为“结业考试”。

```
model=Fclassifier( ).cuda( )  # 配置 GPU 环境的训练效果更佳
loss=nn.CrossEntropyLoss( )
# 使用 Adam 算法进行优化
optimizer=torch.optim.Adam(model.parameters( ), lr=0.001)
num_epoch=30

for epoch in range(num_epoch):
    epoch_start_time=time.time( )
```

```
        train_acc=0.0
        train_loss=0.0
        val_acc=0.0
        val_loss=0.0

        model.train()  # 使模型处于训练模式
        for i, data in enumerate(train_loader):
              optimizer.zero_grad()  # 将模型的梯度归零
              train_pred=model(data[0].cuda())  # 利用 model()函数得到预测的概率分布
              batch_loss=loss(train_pred, data[1].cuda())  # 计算损失
              batch_loss.backward()  # 反向传播计算梯度
              optimizer.step()  # 用模型更新参数值

              train_acc+=np.sum(np.argmax(train_pred.cpu().data.numpy(), axis=1)==
data[1].numpy())
              train_loss+=batch_loss.item()

        model.eval()
        # 处理验证集
        with torch.no_grad():
              for i, data in enumerate(val_loader):
              val_pred=model(data[0].cuda())
                  batch_loss=loss(val_pred, data[1].cuda())
                  val_acc+=np.sum(np.argmax(val_pred.cpu().data.numpy(), axis=1)=
=data[1].numpy())
                  val_loss+=batch_loss.item()

      # 将结果打印出来
      print('[% 03d/% 03d]% 2.2f sec(s)Train Acc:% 3.6f Loss:% 3.6f| Val Acc:% 3.6f
 loss:% 3.6f'% \
      (epoch+1, num_epoch, time.time()- epoch_start_time, \
      train_acc/train_set.__len__(), train_loss/train_set.__len__(), =\
      val_acc/val_set.__len__(), val_loss/val_set.__len__()))
```

在上述代码中，针对分类任务，损失函数使用交叉熵损失函数 CrossEntropyLoss()；训练过程中使用 Adam 优化器，学习率设为 0.001，训练轮数设为 30；同时，在训练过程中打印出所用时间、训练集和验证集的准确率和损失函数值，以便用户观察网络模型在训练过程中准确率等指标的动态变化，从而看出模型在整个过程中的收敛性如何。

网络在训练集上经过初步训练得到训练参数之后，再利用训练集和验证集共同训练，这样做的好处是训练数据增多，模型的训练效果更好。

```
train_val_x=np.concatenate((train_x, val_x), axis=0)
train_val_y=np.concatenate((train_y, val_y), axis=0)
train_val_set=ImgDataset(train_val_x, train_val_y, train_transform)
train_val_loader=DataLoader(train_val_set, batch_size=batch_size, shuffle=True)
```

根据设定加载好的训练集和验证集 train_val_loader，使用上面得到的模型参数接着训练，相当于把上面利用训练集训练后的模型看成预训练模型。设置过程与上面类似。

```
model_best=Fclassifier().cuda()
loss=nn.CrossEntropyLoss()
optimizer=torch.optim.Adam(model_best.parameters(), lr=0.001)
num_epoch=30

for epoch in range(num_epoch):
    epoch_start_time=time.time()
    train_acc=0.0
    train_loss=0.0

    model_best.train()
    for i, data in enumerate(train_val_loader):
        optimizer.zero_grad()
        train_pred=model_best(data[0].cuda())
        batch_loss=loss(train_pred, data[1].cuda())
        batch_loss.backward()
        optimizer.step()

        train_acc+=np.sum(np.argmax(train_pred.cpu().data.numpy(), axis=1)==
data[1].numpy())
        train_loss+=batch_loss.item()

    print('[%03d/%03d]%2.2f sec(s)Train Acc:%3.6f Loss:%3.6f' % \
    (epoch+1, num_epoch, time.time()-epoch_start_time, \
    train_acc/train_val_set.__len__(), train_loss/train_val_set.__len__()))
```

之后，可以利用最终训练好的网络模型 model_best 在测试集上进行预测，可视化地查看所设计的 Fclassifier 食物图片分类器在测试集上的表现效果，并将测试结果存储在 CSV 文件中。

```
test_set=ImgDataset(test_x, transform=test_transform)
test_loader=DataLoader(test_set, batch_size=batch_size, shuffle=False)
model_best.eval()
prediction=[]
```

```
with torch.no_grad():
    for i, data in enumerate(test_loader):
        test_pred=model_best(data.cuda())
        test_label=np.argmax(test_pred.cpu().data.numpy(), axis=1)
        for y in test_label:
          zprediction.append(y)
# 把结果写入 CSV 文件中
with open("predict.csv", 'w') as f:
    f.write('Id, Category\n')
    for i, y in enumerate(prediction):
        f.write('{}, {}\n'.format(i, y))
```

8.4.5 优化改进

本节设计了一个简单的卷积神经网络，用于完成常见的 11 类食物图片的分类任务。为了学有所用，读者可以进一步思考如何对其优化改进。下面提供几个方向。

(1)数据集的变化。之前使用的是 Food-11 数据集，其中只包含 11 个类别的美食，还有许多种美食未包含在内。读者可以尝试切换到 Food-101 数据集，以完成更精细分类的美食图片识别任务。

(2)网络结构的优化。之前设计的食物图片分类器 Fclassifier 仅经历了 5 次“卷积－正规化－激活－池化”操作和 3 层全连接层操作，结构相对简单。是否可以设计一个更深、更优的网络来应对 Food-101 数据集更复杂的食物图片分类任务？答案当然是肯定的！例如，可以尝试使用残差神经网络 ResNet。进一步地，可以将注意力机制(如通道注意力和空间注意力模块)与 ResNet 相结合，以探索达到怎样的效果。

(3)评价指标的多样性。对于图片分类任务，准确率是最基本的评价指标。然而，还可以从更加多样化的角度使用精确率、召回率、F_1 值、接受者操作特征(receiver operating characteristic，ROC)曲线等综合评估模型的性能。

(4)其他方面。读者在进一步学习的过程中，还可以关注预训练模型的合理选择、多种图像增强策略的应用、学习率动态调整的设置、智能优化网络参数等。

从浅入深、从易到难，这条深度学习之路道阻且长。在未来的探索中，读者将有更广阔的发展空间。

习　题

1. 为什么会出现深度神经网络？其与浅层神经网络的主要区别是什么？

2. 简述在卷积神经网络中池化、填充和激活函数的作用。

3. 用表格或示意图的形式，绘制 VGG19 网络的结构(需要展示各层特征图的大小变化)。

4. LSTM 主要由哪些部分组成？各部分的作用是什么？

5. 针对常见的图像识别和自然语言处理任务，可以从哪些方面对网络模型进行改进？

习题解析